

ระบบบริหารการจัดการการขนส่งทั้งระบบ
Transportation Management System

นางสาวอรปรียา พึ่งชู

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ
คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2566

ปริญญานิพนธ์เรื่อง	:	ระบบบริหารการจัดการการขนส่งทั้งระบบ Transportation Management System
โดย	:	นางสาวอรปรียา พึ่งชู
สาขาวิชา	:	วิทยาการคอมพิวเตอร์
ภาควิชา	:	วิทยาการคอมพิวเตอร์และสารสนเทศ
คณะ	:	วิทยาศาสตร์ประยุกต์
อาจารย์ที่ปรึกษา	:	อาจารย์ ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล
ปีการศึกษา	:	2566

คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ อนุมัติให้ปริญญานิพนธ์นี้เป็นส่วนนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์

..... ประธานกรรมการ
(ผู้ช่วยศาสตราจารย์ สถิตย์ ประสมพันธ์)

..... กรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.สุวัจชัย กมลสันติโรจน์)

..... กรรมการ
(อาจารย์ ดร.สรร รัตนสัญญา)

..... กรรมการ
(อาจารย์ ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล)

..... หัวหน้าภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ
(ผู้ช่วยศาสตราจารย์ ดร.อัครา ประโยชน์)

ลิขสิทธิ์ของภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2566

บทคัดย่อ

ปัจจุบัน ทีม Route plan รับผิดชอบในการวางแผนเส้นทาง (Route) สำหรับการจัดส่งสินค้าของบริษัท การทำงานหลัก ๆ ของทีมนี้คือการกำหนดเส้นทางที่เหมาะสม และมีประสิทธิภาพสูงสุดในการส่งสินค้าจากที่ต้นทางไปยังปลายทาง โดยพิจารณาปัจจัยต่าง ๆ เช่น ระยะทาง, เวลา, ค่าใช้จ่าย และเงื่อนไขอื่น ๆ ที่มีผลต่อการจัดส่งสินค้า ในการจัดการทำ Route Plan ทุกครั้งจะมีสถานที่ที่จะไปจัดส่งสินค้าที่จะจัดส่งสินค้าที่ไม่ถูกต้อง และสถานที่ที่จัดส่งไม่มีพิกัดที่ตั้งที่จะไปจัดส่งเป็นจำนวนมาก ทำให้ทางทีมที่จัดการ Route plan ต้องใช้เวลาในการหาพิกัดที่ตั้งที่จะไปจัดส่งเพื่อนำ ไปจัด Route plan จึงเป็นผลทำให้ ทางทีม Route plan ใช้เวลาในการพิกัดที่ตั้งใช้เวลานาน อีกทั้งยังเสียค่าบริการ service ในการไปหาพิกัดที่ตั้งเป็นจำนวนมาก ทำให้บริษัทเกิดค่าใช้จ่ายที่เพิ่มขึ้นด้วย

จากปัญหาข้างต้น ผู้จัดทำโครงการจึงได้ Web Management Shipping Area Master นั้นถูกพัฒนาขึ้นมาเพื่อแก้ไขปัญหาในการจัด Route ของทีม Route Plan ในการจัดส่งสินค้าที่ ออฟฟิศเมท โดยการให้ทีมสามารถจัดการพิกัดสถานที่ได้สะดวกและรวดเร็ว นอกจากนี้ยังช่วยลดเวลา และค่าใช้จ่ายที่เกิดจากการค้นหาพิกัดสถานที่ทุกครั้งที่จัดส่งสินค้าหน้าที่ของ Web Management Location Master คือทำให้ทีม Route plan สามารถตรวจสอบ และแก้ไขพิกัดสถานที่ได้โดยสะดวก ทำให้กระบวนการจัด Route เร็วขึ้น ลดการใช้เวลาและค่าใช้จ่าย เพื่อเพิ่มประสิทธิภาพของทีม และลดความซ้ำซ้อนในการค้นหาพิกัดสถานที่

กิตติกรรมประกาศ

การจัดทำปริญญานิพนธ์เรื่อง Transportation Management System สำเร็จลุล่วงไปด้วยดี เพราะได้นำองค์ความรู้ที่ได้ศึกษาเล่าเรียนจากมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ ที่ซึ่งให้ความรู้ทั้งทางด้านทฤษฎีและปฏิบัติ รวมไปถึงข้อคิดเห็นทางวิชาการต่าง ๆ ที่เป็นประโยชน์อย่างยิ่งมาประยุกต์ใช้งาน

นอกจากมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือแล้ว ยังได้รับความอนุเคราะห์จากบุคคลต่าง ๆ ดังต่อไปนี้ ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล ซึ่งเป็นอาจารย์ที่ปรึกษาปริญญานิพนธ์ที่ได้กรุณาสละเวลาอันมีค่า ให้คำปรึกษา และได้แนะนำแนวทางแก้ปัญหา และข้อผิดพลาดต่าง ๆ ที่เกิดขึ้นขณะที่จัดทำปริญญานิพนธ์ คุณพี่เจ และพี่แน่น รวมถึงท่านคณาจารย์ทั้งในภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ และภายนอกภาควิชา ที่ให้กำลังใจ และให้ความช่วยเหลือ ตลอดจนคอยสั่งสอนถ่ายทอดประสบการณ์ความรู้ จึงขอขอบพระคุณไว้ ณ โอกาสนี้ และขอขอบคุณรุ่นพี่ระดับบัณฑิตศึกษา ที่คอยชี้แนะให้คำแนะนำ ทั้งนี้ผู้จัดทำรู้สึกซาบซึ้งในความกรุณาของทุกท่านเป็นอย่างยิ่ง และขอขอบพระคุณไว้ ณ โอกาสนี้ด้วย

สุดท้ายนี้ ผู้จัดทำขอกราบขอบพระคุณ บิดา มารดา และบุคคลในครอบครัวทุกท่านรวมไปถึงเพื่อน ๆ ทุกคนที่ได้ให้คำปรึกษา คอยช่วยเหลือให้ด้านต่าง ๆ และยังให้กำลังใจมาโดยตลอด ตั้งแต่เริ่มต้นจนปริญญานิพนธ์เล่มนี้ประสบผลสำเร็จไปได้ด้วยดี

นางสาวอรปรียา พึ่งชู

15 พฤศจิกายน 2566

สารบัญ

	หน้า
บทคัดย่อ	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญตาราง	ฉ
สารบัญภาพ	ช
บทที่ 1 บทนำ	
1.1 ความเป็นมาและความสำคัญของโครงงาน	1
1.2 วัตถุประสงค์ของโครงงาน	2
1.3 ขอบเขตโครงงาน	2
1.4 ประโยชน์ที่คาดว่าได้รับ	2
1.5 แผนการดำเนินงาน	3
บทที่ 2 ทฤษฎีที่เกี่ยวข้อง	
2.1 Vue	4
2.2 Node.Js	4
2.3 C#	5
2.4 Maps JavaScript API	6
2.5 JavaScript	6
2.6 SQL	7
2.7 PyThaiNLP	7
2.8 Python	9
2.9 PostgREST	9
2.10 OfficeMate	10
บทที่ 3 ขั้นตอนและวิธีการดำเนินงาน	
3.1 ภาพรวมการทำงานของระบบ	11
3.2 System Architecture Design	12
3.3 การจัดการข้อมูล	14
3.4 การดูข้อมูล	15
3.5 ER Diagram	17
3.6 Data Table	17
3.7 Flowchart	21

สารบัญ (ต่อ)

	หน้า
บทที่ 4 การพัฒนาระบบ	
4.1 เครื่องมือที่ใช้ในการพัฒนาระบบ	22
4.2 Service การจัดการข้อมูลที่อยู่ลูกค้า	22
4.3 การพัฒนาระบบส่วนการแสดงผล	30
4.4 API ที่สร้างให้ระบบเรียกใช้งาน	38
บทที่ 5 ผลการดำเนินโครงงาน	
5.1 ผลการดำเนินงาน	45
5.2 การแสดงผลของ Web Application	45
บทที่ 6 บทสรุปและแนวทางในการพัฒนาต่อ	
6.1 สรุปผลการดำเนินงาน	56
6.2 ปัญหาที่พบระหว่างการดำเนินงาน	56
6.3 แนวทางแก้ปัญหา	56
6.4 แนวทางในการพัฒนาต่อ	57
บรรณานุกรม	58

สารบัญตาราง

	หน้า
ตารางที่ 1-1 ตารางการดำเนินงาน	3
ตารางที่ 3-1 Use Case Description : จัดการข้อมูล Shipping Area Master	14
ตารางที่ 3-2 Use Case Description : ดูข้อมูล	16
ตารางที่ 3-3 แสดงตาราง BuildingMaster เป็นตารางเก็บข้อมูลตึก	17
ตารางที่ 3-4 แสดงตาราง AddressMaster ที่เก็บข้อมูลที่อยู่ของซอยถนนที่เอาไว้ตรวจสอบ	18
ตารางที่ 3-5 แสดงตาราง CustShippingAddress เป็นตารางเก็บข้อมูลที่อยู่ลูกค้า	18
ตารางที่ 3-6 แสดงตาราง ShippingAreaMaster เป็นตารางเก็บข้อมูล Master สถานที่	19

สารบัญภาพ

		หน้า
ภาพที่ 2-1	ภาพการตัดคำ	8
ภาพที่ 2-2	ผลลัพธ์การตัดคำ	8
ภาพที่ 3-1	ภาพรวมการทำงานของระบบ	11
ภาพที่ 3-2	System Architecture Design	12
ภาพที่ 3-3	กระบวนการ Cleansing Data	12
ภาพที่ 3-4	กระบวนการ Migration	13
ภาพที่ 3-5	Use Case Diagram การจัดการข้อมูล	14
ภาพที่ 3-6	Use Case Diagram การดูข้อมูล	15
ภาพที่ 3-7	ER Diagram	17
ภาพที่ 3-8	Flowchart	21
ภาพที่ 4-1	การติดต่อฐานข้อมูล	23
ภาพที่ 4-2	ฟังก์ชัน spilt_input	23
ภาพที่ 4-3	กระบวนการดึงข้อมูล	24
ภาพที่ 4-4	ฟังก์ชัน findRoadandSoi	25
ภาพที่ 4-5	กระบวนการหา Road, Soi, Building, Village	26
ภาพที่ 4-6	การ Validation Road Soi	26
ภาพที่ 4-7	การดึงข้อมูลมาจัดกลุ่มข้อมูล	27
ภาพที่ 4-8	การ Transfer ข้อมูล	28
ภาพที่ 4-9	การเตรียมข้อมูลแบบ bulk	28
ภาพที่ 4-10	การสร้างเนื้อหาของข้อมูล	29
ภาพที่ 4-11	การติดต่อ Service Google Map API	30
ภาพที่ 4-12	การสร้างแผนที่	30
ภาพที่ 4-13	การร้องขอข้อมูลเกี่ยวกับพิกัด	31
ภาพที่ 4-14	การใช้ข้อมูลจาก Geocoding API	32
ภาพที่ 4-15	การร้องขอ Address จากพิกัด	33
ภาพที่ 4-16	การสร้าง Google Map Marker	34
ภาพที่ 4-17	การร้องขอฐานข้อมูล	34
ภาพที่ 4-18	การกำหนดค่า item	35
ภาพที่ 4-19	การนำข้อมูลจากฐานข้อมูลมาแสดงผล	36
ภาพที่ 4-20	การสร้าง InfoWindow	37
ภาพที่ 4-21	การเรียกใช้ InfoWindow	37

สารบัญภาพ (ต่อ)

		หน้า
ภาพที่ 4-22	GetLocationMasterSelectAsync	38
ภาพที่ 4-23	GetLocationMasterListAsync	39
ภาพที่ 4-24	UpdateLocationMastertAsync	41
ภาพที่ 4-25	CreateLocationMastertAsync	43
ภาพที่ 5-1	หน้าจอเว็บไซต์ Web Management	45
ภาพที่ 5-2	Criteria การกรองข้อมูล	45
ภาพที่ 5-3	หน้าจอเลือกแสดง Display	46
ภาพที่ 5-4	หน้าจอแสดงแบบ Address	46
ภาพที่ 5-5	หน้าจอแสดงพิกัดข้อมูล	47
ภาพที่ 5-6	หน้าจอแสดงแบบ Google Map	47
ภาพที่ 5-7	หน้าจอแสดงข้อมูลพิกัดบนแผนที่	48
ภาพที่ 5-8	หน้าจอข้อมูลเริ่มต้น	48
ภาพที่ 5-9	หน้าจอข้อมูลหลังมีการลาก Marker	49
ภาพที่ 5-10	หน้าจอแสดงการรอยืนยันการ Update	49
ภาพที่ 5-11	หน้าจอแสดงผลการอัปเดตสำเร็จ	50
ภาพที่ 5-12	หน้าจอแสดงการ Update	50
ภาพที่ 5-13	หน้าจอข้อมูลเริ่มต้น	50
ภาพที่ 5-14	หน้าจอแสดงการแก้ไขข้อมูล	51
ภาพที่ 5-15	หน้าจอแสดงผลการแก้ไขมูลสำเร็จ	51
ภาพที่ 5-16	หน้าจอข้อมูลเริ่มต้น	52
ภาพที่ 5-17	หน้าจอแสดงข้อมูล Check Location	52
ภาพที่ 5-18	หน้าจอแสดงปุ่ม Back	53
ภาพที่ 5-19	หน้าจอแสดงข้อมูลพิกัดเริ่มต้น	53
ภาพที่ 5-20	info window การกดที่แผนที่	54
ภาพที่ 5-21	หน้าจอแสดงการเพิ่มข้อมูล	55
ภาพที่ 5-22	หน้าจอแสดงผลการเพิ่มข้อมูลสำเร็จ	55

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

บริษัท ออฟฟิศเมท (ไทย) เป็นบริษัทที่ขายอุปกรณ์สำนักงาน และสินค้าเพื่อธุรกิจที่พร้อมตอบโจทย์ธุรกิจและการจัดซื้อ โดยมีระบบการสั่งซื้อออนไลน์ที่สะดวก และมีการจัดการสินค้า และการจัดส่งของ ที่มีระบบ Transportation Management System (TMS) มีบทบาทสำคัญในการบริหารจัดการกระบวนการขนส่งค้า และการจัดส่งสินค้าให้กับลูกค้าของบริษัท ในปัจจุบันทางบริษัท ออฟฟิศเมท (ไทย) มีรูปแบบการขนส่ง 2 รูปแบบ คือ 1. จัดส่งเอง (Own fleet) และ 2. Logistics Service Provider (Third Party) เช่น Kerry, Flash โดยทางทีม Route Planning Management ที่ทางผู้จัดทำได้มีโอกาสไปร่วมงาน มีบทบาทในการจัดการกระบวนการขนส่ง และการวางแผนการใช้ยานพาหนะในส่วนของ Own fleet ซึ่งเป็นส่วนหนึ่งของโครงสร้างการขนส่งของบริษัท

ทีม Route Planning Management ที่จัดการในส่วนนี้อยู่เกิดปัญหาในส่วนของการจัดการละติจูดและลองจิจูด ส่วนแรกคือ ปริมาณงานการจัดส่งที่มาในระบบยังไม่มีข้อมูล ละติจูดและลองจิจูด มีจำนวนมากในส่วนของพื้นที่กรุงเทพและปริมณฑล ปัญหานี้ทำให้กระบวนการวางแผนการจัดส่งเริ่มต้นด้วยการหาข้อมูลพิกัด ละติจูดและลองจิจูด ซึ่งอาจทำให้ เสียเวลา และทรัพยากรมากขึ้น ส่วนต่อมาคือ การหาข้อมูล ละติจูดและลองจิจูด ในงานที่ไม่มีข้อมูลล่วงหน้า สามารถเสียเวลามากขึ้น ซึ่งอาจทำให้กระบวนการวางแผนการจัดส่งช้าลง และมีผลกระทบต่อเวลาการจัดส่งที่มีประสิทธิภาพ ส่วนสุดท้ายคือ ค่าใช้จ่ายในการเรียกใช้งาน Google Service API เนื่องจากข้อมูลการจัดส่ง ละติจูดและลองจิจูดขาดหายมากขึ้นการหาข้อมูลนี้ต้องใช้บริการ Google Service API ต่าง ๆ ซึ่งเพิ่มค่าใช้จ่ายในการไปดึงข้อมูลพิกัด ละติจูดและลองจิจูด

เนื่องจากปัญหานี้โครงงาน Transportation Management System มีความสำคัญในการแก้ไขปัญหาข้อมูล ละติจูดและลองจิจูดที่ขาดหาย และลดค่าใช้จ่ายในการใช้ Google Service API โดยเฉพาะในพื้นที่กรุงเทพและปริมณฑล การจัดการข้อมูลเหล่านี้ และลดเวลาในการหาข้อมูลมีความสำคัญ เพื่อที่จะเพิ่มประสิทธิภาพในกระบวนการจัด Route การจัดส่ง และควบคุมค่าใช้จ่ายเพื่อให้การขนส่งเป็นไปอย่างมีประสิทธิภาพ

1.2 วัตถุประสงค์ของโครงงาน

เพื่อพัฒนา Web Management Shipping Area Master สำหรับเพิ่มประสิทธิภาพในการจัด Route ส่งสินค้า โดยลดเวลาและค่าใช้จ่าย ด้วยการให้ทีม Route plan ใช้เครื่องมือนี้ในการจัดการและปรับปรุงพิกัดสถานที่อย่างสะดวก ทำให้กระบวนการจัด Route เร็วขึ้นและลดความซ้ำซ้อน อีกทั้งช่วยลดข้อผิดพลาด และเพิ่มความยืดหยุ่นในการปรับปรุงกระบวนการจัด Route ส่งสินค้า

1.3 ขอบเขตโครงงาน

1.3.1 สร้างฐานข้อมูลสำหรับใช้ในการจัดการพิกัดสถานที่

1.3.2 สร้าง Service จัดการข้อมูลที่อยู่ของลูกค้า ให้เป็น Shipping Area Master

1.3.2.1 Service สามารถจัดการข้อมูลอยู่ของลูกค้า ให้มีความละเอียด และถูกต้องมากกว่าเดิม

1.3.2.2 Service สามารถจัดการแบ่งข้อมูลอยู่ของลูกค้า ให้มีความละเอียดขึ้น เรื่องของซอย ถนน อาคาร หมู่บ้าน

1.3.2.3 Service สามารถตรวจสอบข้อมูล ซอย ถนน อาคาร หมู่บ้าน ว่ามีอยู่จริงและมีความถูกต้อง

1.3.3 พัฒนา Web Application สำหรับจัดการข้อมูลพิกัดสถานที่ในเว็บ Transportation Management System ให้สามารถใช้งานกับทีม Route Planning Management

1.3.3.1 ระบบแสดงข้อมูล Shipping Area Master เพื่อให้ทางทีม Route Planning Management สามารถเรียกดูได้

1.3.3.2 ระบบตรวจสอบข้อมูล Shipping Area Master

1.3.3.3 ระบบสร้างข้อมูล Shipping Area Master

1.3.4.4 ระบบแก้ไขข้อมูล Shipping Area Master

1.4 ประโยชน์ที่คาดว่าได้รับ

1.4.1 ได้ Web Application ที่ใช้สำหรับการจัดการ Shipping Area Master ที่ตอบโจทย์กับทางทีม Route planning ในการจัด Route plan

1.4.2 ได้ Service Python ที่เอาไว้ใช้งานในการจัดการข้อมูล ที่อยู่ลูกค้า ให้มีความละเอียดและถูกต้องมากขึ้น ทำให้ทางทางทีม Route Planning Management ได้รับข้อมูล ละติจูด และลองจิจูดที่ขาดหายหายไป เพิ่มมากขึ้น ทำให้เพิ่มความถูกต้องของข้อมูล

1.4.3 ช่วยลดค่าใช้จ่ายในการเรียกใช้ Google Service API ทำให้ช่วยบริษัทลดค่าใช้จ่ายในการดึงข้อมูลพิกัด ละติจูดและลองจิจูด ซึ่งเป็นประโยชน์ทางธุรกิจ และสามารถนำกำไรนั้นลดทรัพยากรบุคคลของบริษัทได้

1.4.4 การเพิ่มประสิทธิภาพในการจัดส่งจะช่วยบริษัทให้บริการลูกค้าด้วยเวลาการจัดส่งที่มีคุณภาพและมีประสิทธิภาพมากยิ่งขึ้น

1.5 แผนการดำเนินงาน

ตารางที่ 1-1 ตารางการดำเนินงาน

แผนการทำงานแต่ละสัปดาห์	ระยะเวลาการดำเนินงาน						
	พ.ค	มิ.ย	ก.ค	ส.ค	ก.ย	ต.ค	พ.ย
Planning							
1. คุยเรื่อง Scope Project							
2. ทำความเข้าใจระบบที่จะเข้าไปทำ							
3. เรียนรู้ Tool ที่บริษัทจะให้ใช้ใน Project							
4. เตรียมหัวข้อ Present ให้กับทางมหาลัย							
Development							
1. วิเคราะห์และออกแบบฐานข้อมูลระบบ							
2. Data Cleansing							
3. Migration							
4. ออกแบบและพัฒนาหน้าเว็บ							
5. Develop Shipping Area Master							
6. นำฐานข้อมูลมาใช้กับ Web							
7. นำเสนอระบบกับผู้ใช้งาน							
8. ทดสอบและปรับปรุงระบบ							
Presentation and Document							
1. Presentation							
2. Project Document							

บทที่ 2
ทฤษฎีที่เกี่ยวข้อง

2.1 Vue

Vue คือ เป็นFramework JavaScript แบบไดนามิกสำหรับสร้างส่วนต่อประสานผู้ใช้ ระบบนี้เป็นที่รู้จักกันดีในเรื่องของช่วงการเรียนรู้ที่สูงชัน อย่างไรก็ตามมันเป็นเฟรมเวิร์คที่เข้าถึงได้ง่ายและตรงไปตรงมา ซึ่งเราอาจเริ่มสร้างเว็บแอปใน Vue.js ด้วยความเข้าใจพื้นฐานของ HTML, CSS และ JavaScript เท่านั้น เส้นโค้งการเรียนรู้สั้น ๆ เป็นคุณลักษณะของกรอบการทำงานแบบก้าวหน้านี้ มีความยืดหยุ่น และสามารถใช้เป็นยูทิลิตี้หรือเฟรมเวิร์คที่ครอบคลุมในการพัฒนาเว็บ ใช้สำหรับพัฒนาเว็บแอปพลิเคชันแบบ Single Page Application (SPA)

โดย Vue.js นำเสนอแนวคิดของ Reactive Data Binding ที่ช่วยให้ข้อมูลที่แสดงผลบนหน้าเว็บสามารถปรับปรุงอัตโนมัติเมื่อมีการเปลี่ยนแปลงในข้อมูลต้นฉบับ ซึ่งทำให้การจัดการข้อมูล และการแสดงผลเป็นไปอย่างมีประสิทธิภาพ และกระชับ Component-Based Architecture เป็นแนวคิดที่ Vue.js นำเข้ามาจากแนวคิดของการสร้าง UI ในรูปแบบของ Component ที่สามารถนำไปใช้ซ้ำได้ แต่ละ Component มีส่วนประกอบที่มีการกำหนดค่าและติดต่อกันได้ ทำให้การพัฒนาและบริหารจัดการโค้ดเป็นไปอย่างมีระเบียบและยืดหยุ่น ทฤษฎีของ Virtual DOM ทำให้ Vue.js มีประสิทธิภาพในการอัพเดทและแสดงผลข้อมูลบนหน้าเว็บ โดย Virtual DOM ทำหน้าที่เปรียบเทียบและอัพเดทเพียงส่วนที่มีการเปลี่ยนแปลง ซึ่งช่วยลดการใช้ทรัพยากร และเพิ่มประสิทธิภาพการทำงานการให้ความสำคัญกับการทำงานแบบ Reactive Programming ทำให้ Vue.js มีความสามารถในการตอบสนองกับเหตุการณ์ และการเปลี่ยนแปลงข้อมูลอย่างทันที ซึ่งเป็นที่นิยมในการพัฒนาแอปพลิเคชันที่มีประสิทธิภาพและสมบูรณ์

ทั้งนี้ Vue.js ยังมีประสิทธิภาพสูงในการทำงานร่วมกับ API และมีการสนับสนุนทั้งในด้านการทำงานแบบ Server-Side Rendering (SSR) และ Client-Side Rendering (CSR) ทำให้เหมาะสำหรับโปรเจคที่ต้องการความยืดหยุ่นในการเลือกใช้วิธีการ rendering และมีคุณสมบัติที่ทำให้การพัฒนาเว็บแอปพลิเคชันเป็นไปอย่างมีประสิทธิภาพและสะดวกสบาย

2.2 Node.Js

Node.js เป็นเทคโนโลยีที่มีผลกระทบมหาศาลในโลกของพัฒนาซอฟต์แวร์ เดิมทีนั้น JavaScript มักถูกใช้ในบริบทของภาษาสคริปต์ที่ทำงานในบราวเซอร์เท่านั้น แต่ด้วยการนำ Node.js

เข้ามาเสริมทัพในโดเมนของ server-side, JavaScript กลายเป็นทัศนคติในการพัฒนาแอปพลิเคชันที่มีประสิทธิภาพสูงและตอบสนองได้รวดเร็ว

ทฤษฎีหลักของ Node.js มีความเฉพาะตัว โดยทำงานตามหลักการ Non-blocking I/O, Event-driven และ Single-threaded ทำให้สามารถจัดการกับการทำงานแบบ asynchronous ได้โดยมีประสิทธิภาพสูง ทำให้สามารถจัดการกับ concurrent connections ได้พร้อมกันโดยไม่ส่งผลกระทบต่อประสิทธิภาพของระบบ เรื่องสำคัญที่ Node.js ทำให้เด่นชัดคือการให้ความสำคัญกับการทำงานแบบ event-driven ทำให้การตอบสนองต่อเหตุการณ์ต่าง ๆ เป็นไปอย่างมีประสิทธิภาพ นอกจากนี้ Node.js ยังได้รับความนิยมในการสร้างแอปพลิเคชันแบบ real-time เช่นแอปพลิเคชันแชทหรือการสื่อสารreal time Node.js ยังถือเป็นแพลตฟอร์มที่ cross-platform ทำให้นักพัฒนาสามารถทำงานบนหลายระบบปฏิบัติการได้ ทำให้มีความยืดหยุ่นในการพัฒนาและใช้งาน การใช้งาน Node.js ยังมีประโยชน์ในด้านการสร้างแอปพลิเคชันที่มีประสิทธิภาพสูงทั้งในด้านการจัดการ I/O operations การทำงานกับฐานข้อมูล และการสร้าง API ด้วยคุณสมบัติและประโยชน์ทั้งหลายนี้ Node.js กลายเป็นเครื่องมือที่สำคัญในโลกของพัฒนาซอฟต์แวร์ และมีผลกระทบใหญ่ในการเปลี่ยนแปลงแนวคิดในการพัฒนาแอปพลิเคชันบนเว็บ

2.3 C#

C# เป็นภาษาโปรแกรมที่มีรูปแบบการเขียนโค้ดที่เน้นทฤษฎีแบบ Object-Oriented Programming (OOP) โดยใช้ Class และ Object เพื่อจัดการข้อมูลและฟังก์ชัน โดยมีคุณสมบัติที่สำคัญอย่าง Inheritance ที่ช่วยในการสืบทอดคุณสมบัติของ Class, Encapsulation ที่สนับสนุนการกลบกล้องข้อมูลและฟังก์ชัน และ Polymorphism ที่ช่วยให้สามารถใช้งาน Method หรือ Object ในรูปแบบต่าง ๆ ได้ นอกจากนี้ C# ยังมีคุณสมบัติอื่น ๆ เช่น Exception Handling สำหรับการจัดการข้อผิดพลาด Delegates และ Events ที่ช่วยในการจัดการกับการเหตุการณ์ LINQ ที่ให้ความสะดวกในการจัดการข้อมูล ระบบ Asynchronous Programming ที่ช่วยในการทำงานพร้อมกันได้ Generic Programming ที่สนับสนุนการใช้โค้ดกับประเภทข้อมูลต่าง ๆ และระบบ Garbage Collection ที่จัดการหน่วยความจำเพื่อลดความซับซ้อนในการจัดการขยะ (Garbage) ทั้งหมดนี้ทำให้ C# เป็นภาษาที่มีประสิทธิภาพสูง และเหมาะสำหรับการพัฒนาโปรแกรมที่เชื่อถือได้ และมีความยืดหยุ่น

2.4 Maps JavaScript API

Maps JavaScript API เป็นส่วนหนึ่งของ Google Maps Platform ที่ช่วยให้นักพัฒนาสามารถนำเอาแผนที่และข้อมูลทางภูมิศาสตร์มาใช้งานในเว็บไซต์หรือแอปพลิเคชันของตนได้อย่างง่ายดายและน่าสนใจ ทฤษฎีที่เกี่ยวข้องกับ Maps JavaScript API นั้น ล้วนเป็นส่วนสำคัญที่นักพัฒนาควรทราบ Maps JavaScript API ใช้เทคโนโลยีทางเว็บเบราว์เซอร์เพื่อแสดงแผนที่ และข้อมูลทางภูมิศาสตร์ในหน้าเว็บไซต์หรือแอปพลิเคชัน ซึ่งทำให้สามารถทำงานได้ทั้งบนเบราว์เซอร์ทุกประเภทโดยไม่ต้องติดตั้งโปรแกรมเสริมเพิ่ม API นี้ช่วยให้นักพัฒนาสามารถแสดงแผนที่ เพิ่มหมุด สามารถดูทิศทาง แสดงข้อมูลทางภูมิศาสตร์ หรือปรับแต่งรูปแบบแผนที่ตามความต้องการของโปรแกรม

การใช้งาน Maps JavaScript API ต้องใช้ API key เพื่อทำให้ Google Maps รู้ว่าใครกำลังใช้งาน และควรให้สิทธิ์การเข้าถึงอย่างไร Maps JavaScript API ให้ความยืดหยุ่นในการปรับแต่งสไตล์แผนที่, การเพิ่มข้อมูลที่กำหนดเอง หรือการปรับแต่งการตอบสนองของแผนที่ นอกจากนี้ API ยังมีฟีเจอร์การให้บริการข้อมูลทางภูมิศาสตร์อื่น ๆ ที่เกี่ยวข้อง เช่น Street View, Elevation และ Time Zone การใช้ Maps JavaScript API มีความสำคัญในการพัฒนาแอปพลิเคชันที่เกี่ยวข้องกับตำแหน่งทางภูมิศาสตร์ การนำทาง หรือแสดงข้อมูลที่เชื่อมโยงกับพื้นที่บนแผนที่ ทฤษฎีและความเข้าใจที่ดีเกี่ยวกับ Maps JavaScript API จะช่วยให้นักพัฒนาสามารถนำ เอาฟีเจอร์ที่มีให้ใน Google Maps Platform มาประยุกต์ใช้ในโปรแกรมของตนได้อย่างมีประสิทธิภาพ

2.5 JavaScript

JavaScript เป็นภาษาโปรแกรมที่ได้รับความนิยม และได้รับการใช้งานอย่างแพร่หลายในพัฒนาเว็บแอปพลิเคชัน ทฤษฎี JavaScript เกี่ยวกับหลักการพื้นฐาน และลักษณะที่ทำให้มันเป็นที่นิยมมีดังนี้

JavaScript เป็นภาษาโปรแกรมที่ทำงานบนเว็บเบราว์เซอร์ และสามารถประมวลผลโค้ดได้ทันทีในเวลาที่ผู้ใช้เข้าถึงหน้าเว็บ ภาษานี้ถูกออกแบบมาเพื่อทำให้การโปรแกรมเมอร์สามารถเขียนโค้ดที่ทำงานได้ทั้งฝั่ง client และ server JavaScript เป็นภาษาที่มีลักษณะ Dynamic Typing ซึ่งหมายความว่าไม่จำเป็นต้องระบุประเภทของตัวแปรล่วงหน้า นี้ทำให้ JavaScript เป็นภาษาที่ยืดหยุ่นและให้ความสามารถในการเปลี่ยนประเภทของข้อมูลได้ JavaScript มีระบบการจัดการ Event ที่ทำให้สามารถตอบสนองต่อการเกิดเหตุการณ์ต่าง ๆ บนหน้าเว็บได้อย่างสมบูรณ์นักพัฒนาสามารถเชื่อมโยงฟังก์ชันหรือโค้ดกับเหตุการณ์เหล่านี้เพื่อสร้างประสบการณ์ผู้ใช้ที่มีการตอบสนอง JavaScript เป็นภาษาที่สนับสนุนการโปรแกรมแบบ Asynchronous ซึ่งทำให้สามารถทำงานพร้อม

กันหลายกิจกรรมโดยไม่ต้องรอให้กิจกรรมก่อนหน้าเสร็จสิ้นนักพัฒนาสามารถใช้ Callbacks, Promises, Async/Await เพื่อจัดการกับกระบวนการ Asynchronous โดย JavaScript เป็นภาษาที่ต้องการให้นักพัฒนาให้ความสำคัญกับความปลอดภัย เนื่องจากมีความเสี่ยงในการโจมตี XSS (Cross-Site Scripting) นักพัฒนาต้องระมัดระวัง และปฏิบัติตามหลักการปฏิบัติที่ดีเพื่อป้องกันปัญหาด้านความปลอดภัยนี้

2.6 SQL

SQL หรือ Structured Query Language เป็นภาษาโปรแกรมที่ถูกออกแบบมาเพื่อจัดการข้อมูลในฐานข้อมูล ในทฤษฎี SQL การทำงานกับข้อมูลจะเกิดในรูปแบบของตารางที่ถูกสร้างขึ้นโดยใช้คำสั่ง CREATE TABLE ตารางนี้จะประกอบไปด้วยคอลัมน์ต่าง ๆ ที่กำหนดประเภทข้อมูลและข้อกำหนดต่าง ๆ เพื่อเพิ่มข้อมูลในตาราง นักพัฒนาจะใช้คำสั่ง INSERT INTO ซึ่งจะระบุชื่อตารางและข้อมูลที่ต้องการเพิ่ม การค้นหาข้อมูลจะใช้คำสั่ง SELECT โดยสามารถกำหนดเงื่อนไขเพื่อกรองข้อมูลที่ต้องการ การปรับปรุงข้อมูลในตารางจะใช้คำสั่ง UPDATE และการลบข้อมูลจะใช้คำสั่ง DELETE SQL ยังมีความสามารถในการจัดการคีย์ เช่น Primary Key ที่ใช้ระบุแถวที่ไม่ซ้ำกันในตาราง และ Foreign Key เพื่อเชื่อมโยงข้อมูลระหว่างตาราง ระบบการให้สิทธิ์ทำให้นักพัฒนาสามารถกำหนดสิทธิ์การเข้าถึง และดำเนินการบนข้อมูลได้อย่างควบคุม ดังนั้น SQL มีบทบาทสำคัญในการพัฒนาแอปพลิเคชันที่ใช้งานกับข้อมูล ทำให้นักพัฒนาสามารถบริหารจัดการข้อมูลในรูปแบบที่มีระบบ ค้นหา และปรับปรุงได้อย่างมีประสิทธิภาพ

2.7 PyThaiNLP

PyThaiNLP เป็นไลบรารีที่สำคัญสำหรับประมวลผลภาษาไทยในภาษา Python ซึ่งมีความสามารถที่หลากหลาย เริ่มต้นจากการตัดคำ (word segmentation) การแยกประโยค (sentence segmentation) และการตัดสระ (syllable segmentation) ทำให้นักพัฒนาสามารถดำเนินการกับข้อมูลภาษาไทยได้อย่างมีประสิทธิภาพ นอกจากนี้ PyThaiNLP ยังมีฟังก์ชันที่เกี่ยวข้องกับการประมวลผลภาษาธรรมชาติอื่น ๆ เช่น Named Entity Recognition (NER) ที่ช่วยในการจดจำและแยกแยะองค์กร สถานที่ และบุคคลในข้อความ รวมถึงการสนับสนุนการแก้ไขคำผิดและ Spell Checking

เนื่องจาก PyThaiNLP มีความยืดหยุ่นสูง ทำให้เราสามารถนำไปใช้ร่วมกับ Machine Learning (ML) ในหลายด้าน เช่น การทำ Transfer Learning โดยนำ Pre-trained Models มาใช้ในงานต่าง ๆ ทำให้การประมวลผลข้อความภาษาไทยด้วย ML เป็นไปได้มากยิ่งขึ้น นอกจากนี้

PyThaiNLP ยังสามารถใช้ในขั้นตอนการเตรียมข้อมูล (preprocessing) และ Feature Engineering ทำให้สะดวกต่อการสร้างและปรับปรุงโมเดล ML ในงานที่เกี่ยวข้อง

ดังนั้น PyThaiNLP เป็นเครื่องมือสำคัญที่นักพัฒนาและนักวิจัยสามารถใช้เพื่อทำให้กระบวนการประมวลผลภาษาไทยและการใช้ ML กลายเป็นไปอย่างมีประสิทธิภาพ และสามารถปรับเปลี่ยนได้ตามความต้องการ เช่นการตัดคำ

2.7.1 การตัดคำ

```
from pythainlp import word_tokenize

text = "ทดสอบการตัดคำภาษาไทย"
proc = word_tokenize(text, engine='newmm')
print(proc)
```

ภาพที่ 2-1 ภาพการตัดคำ

สำหรับ method การตัดคำ ดังภาพ 2-1 คือ word_tokenize โดย parameter มีใส่ 2 ตัว word_tokenize(text,engine='ระบบตัดคำ') text คือ ข้อความที่ต้องการตัด engine คือระบบตัดคำ มีดังนี้

1. icu — engine ตัวดั้งเดิมของ PyThaiNLP (ความแม่นยำต่ำ) และเป็นค่าเริ่มต้น
2. dict — เป็นการตัดคำโดยใช้พจานุกรมจาก thaiword.txt ใน corpus (ความแม่นยำปานกลาง) จะคืนค่า False หากข้อความนั้นไม่สามารถตัดคำได้
3. longest-matching ใช้ Longest matching ในการตัดคำ
4. mm — ใช้ Maximum Matching algorithm ในการตัดคำภาษาไทย — API ชุดเก่า
5. newmm — ใช้ Maximum Matching algorithm ในการตัดคำภาษาไทย
6. pylexto ใช้ LexTo ในการตัดคำ โดยเป็น Longest matching

```
['ทดสอบ', 'การ', 'ตัด', 'ตำ', 'ภาษาไทย']
```

ภาพที่ 2-2 ผลลัพธ์การตัดคำ

2.8 Python

Python เป็นภาษาโปรแกรมคอมพิวเตอร์ระดับสูง ที่มีจุดเด่นคือ เป็นสคริปต์ภาษาที่ไม่ซับซ้อน เกิดจากการนำข้อดีของภาษาต่าง ๆ มารวมไว้ด้วยกัน ออกแบบมาให้มีโครงสร้างและไวยากรณ์ของภาษาให้มีความง่าย อีกทั้งยังมีไวยากรณ์ที่ช่วยให้เขียนโค้ดสั้นกว่าภาษาอื่น ๆ อีกด้วย โดยเฉพาะหากนำ Python ไปเทียบกับ Java ที่เป็นอีกหนึ่งภาษาโปรแกรมยอดนิยมแล้ว ถือว่าง่ายกว่ามาก ในขณะที่ Python สามารถเขียนโปรแกรมให้จบภายใน 5 บรรทัด แต่ Java อาจต้องใช้ถึง 10 บรรทัด

Python ทำงานแบบ Interpreter ด้วยการแปลชุดคำสั่งทีละบรรทัด เพื่อป้อนเข้าสู่ระบบประมวลผลให้คอมพิวเตอร์ทำงานตามคำสั่ง และยังสามารถนำไปใช้เขียนโปรแกรมได้หลากหลาย หรือที่เรียกว่าเป็น General-purpose language เป็นภาษาที่ไม่ได้ถูกจำกัด และด้วยข้อดีของ Python จึงทำให้ Python ได้รับความนิยมไปใช้กันอย่างแพร่หลาย

Python มีข้อดีอะไรบ้างที่ทำให้หลายคนเลือกใช้ Python เป็นภาษาโปรแกรมที่มีลักษณะ "high-level" ซึ่งหมายความว่าการเขียนโปรแกรมใน Python ไม่ต้องให้ความสำคัญกับรายละเอียดทางเทคนิคมากนัก ซึ่งทำให้เหมาะสำหรับนักพัฒนาทุกระดับ นอกจากนี้ Python ยังเป็นภาษา "interpreted" ซึ่งหมายความว่าไม่จำเป็นต้องคอมไพล์ก่อนที่จะรันโปรแกรม นั้นทำให้การทดสอบและพัฒนาโปรแกรมเป็นไปอย่างรวดเร็ว ยังเป็นภาษาที่สนับสนุนหลายแพลตฟอร์ม และมีความยืดหยุ่นในการใช้งานในหลายด้าน เช่น พัฒนาเว็บไซต์ (web development) การวิทยาศาสตร์ข้อมูล (data science) การประมวลผลข้อความ (natural language processing) และอื่น ๆ ทำให้เป็นภาษาโปรแกรมที่หลากหลายและน่าสนใจตลอดเวลา

2.9 PostgREST

PostgREST เป็นเครื่องมือที่สร้าง API RESTful โดยอัตโนมัติจากฐานข้อมูล PostgreSQL ทำให้การสร้าง และจัดการ API กับฐานข้อมูลเป็นเรื่องง่ายและมีประสิทธิภาพมากขึ้น ให้ผู้พัฒนาสามารถสร้าง API ได้โดยไม่ต้องเขียนโค้ดเพิ่มเติม โดยอัตโนมัติจากโครงสร้างของฐานข้อมูล PostgreSQL ทำให้กระบวนการนี้เป็นไปอย่างรวดเร็ว และน้อยลงในการจัดการ PostgREST สามารถให้บริการ CRUD (Create, Read, Update, Delete) บนข้อมูลใน PostgreSQL ผ่าน HTTP RESTful API ซึ่งทำให้การเข้าถึงข้อมูล และทำการปรับแต่งข้อมูลในฐานข้อมูลเป็นเรื่องง่าย นอกจากนี้ PostgREST ยังมีความสามารถในการจัดการระบบความปลอดภัยและการจัดการสิทธิ์เข้าถึงข้อมูล ทำให้เป็นเครื่องมือที่มีประสิทธิภาพสำหรับการพัฒนา และการจัดการ API ในโครงการที่ใช้ฐานข้อมูล PostgreSQL อยู่แล้ว เรื่องระบบความปลอดภัยเช่นการจัดการการรับรองตัวตน และการจัดการสิทธิ์

เข้าถึงข้อมูลก็เป็นส่วนหนึ่งที่ PostgREST ได้ให้ความสนใจ เพื่อให้นักพัฒนาสามารถควบคุมและปรับแต่งการเข้าถึงข้อมูลได้อย่างเหมาะสม

ดังนั้น PostgREST เป็นเครื่องมือที่มีประสิทธิภาพสำหรับการพัฒนาแอปพลิเคชันที่ต้องการเชื่อมต่อกับฐานข้อมูล PostgreSQL ผ่าน API ที่สร้างขึ้นโดยอัตโนมัติ และมีความสามารถในการจัดการความปลอดภัยของข้อมูลที่ดี

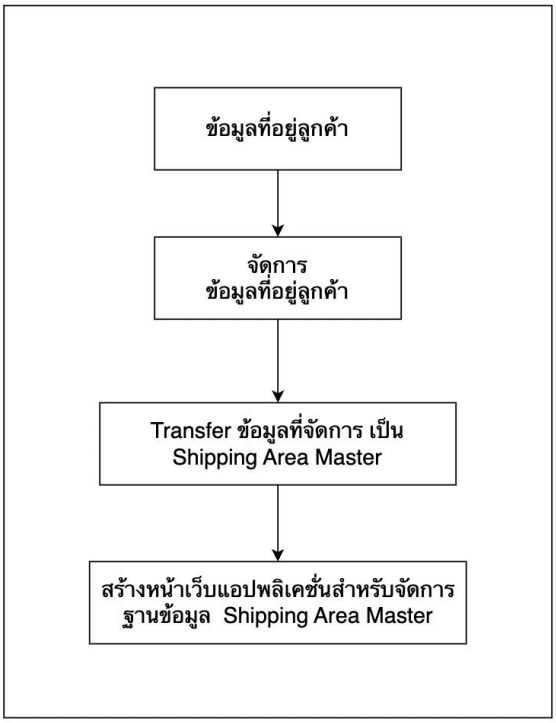
2.10 OfficeMate

Office Mate เป็นบริษัทที่เป็นผู้นำในด้านการจัดหาวัสดุอุปกรณ์สำนักงานและอุปกรณ์สำนักงาน บริษัท Office Mate มีบทบาทสำคัญในการนำเสนอและจัดหาผลิตภัณฑ์ที่เกี่ยวข้องกับการทำงานในสถานที่สำนักงาน Office Mate ยังมีการตลาดออนไลน์ที่ทันสมัย, ทำให้ลูกค้าสามารถสั่งซื้อผลิตภัณฑ์ได้ผ่านเว็บไซต์ของพวกเขา ทำให้การซื้อสินค้าเป็นไปอย่างสะดวกสบาย รวมถึงการให้บริการลูกค้าที่ดีเยี่ยมและการจัดส่งที่รวดเร็ว เนื่องจากดิฉันได้รับผิดชอบโปรเจคในทีม ERP ของบริษัท มีหน้าที่ทำงานกับระบบ Transportation Management System (TMS) โดยมีบทบาทสำคัญในการบริหารจัดการกระบวนการขนส่งค้าและการจัดส่งสินค้าให้กับลูกค้าของบริษัท โดยระบบนี้บริหารจัดการการขนส่งทั้งระบบ ตั้งแต่รับข้อมูลงานจัดส่ง บริหารจัดการเที่ยวรถ ส่งงานและรายละเอียดให้คนขับ ติดตามสถานะการขนส่งแบบ Real Time ทั้งบริษัทขนส่ง และลูกค้ารวมถึงการบันทึกค่าขนส่ง โดยระบบนี้มี Feature สำคัญคือ Route Planning Management โดยมีทีม Route เป็นคนจัดการ คือการวางแผนการใช้ยานพาหนะในส่วนของ Own fleet ซึ่งเป็นส่วนหนึ่งของโครงสร้างการขนส่งของบริษัท กิจกรรมที่ทีมนี้ทำมีการดำเนินการคือ รับข้อมูลที่อยู่การจัดส่งของลูกค้า จัดการหา ละติจูดและลองจิจูด เพื่อที่จะนำไปจัดเส้นทาง และไปการวางแผนการใช้ยานพาหนะ โดยใช้ Google Map และ polygon เพื่อช่วยให้การจัดเส้นทางเป็นอย่างมีประสิทธิภาพและเรียลไทม์และดิฉันได้มาพัฒนาในส่วนของจัดการหา ละติจูดและลองจิจูด

บทที่ 3
ขั้นตอนและวิธีการดำเนินงาน

ในบทนี้จะอธิบายถึงการวิเคราะห์และการออกแบบพัฒนาระบบเว็บแอปพลิเคชันจัดการข้อมูล Shipping Area Master

3.1 ภาพรวมการทำงานของระบบ

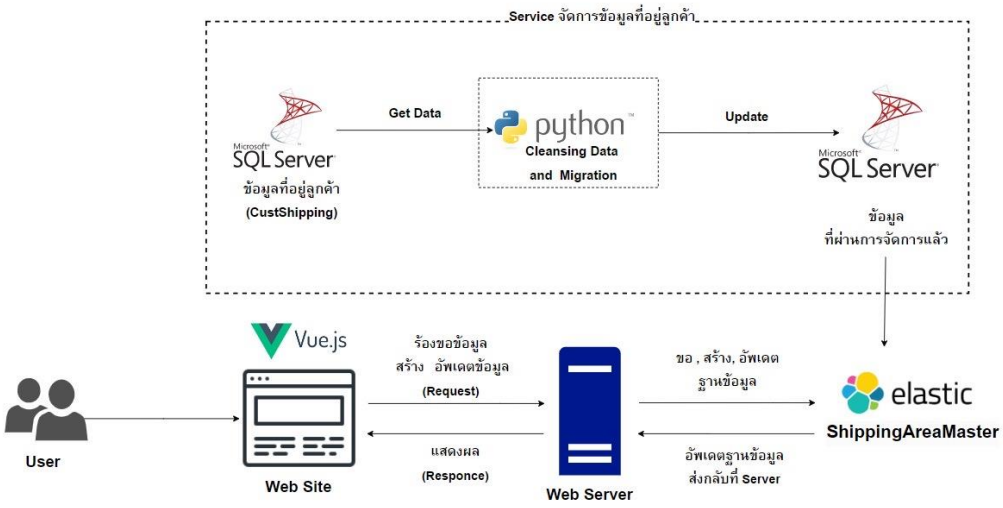


ภาพที่ 3-1 ภาพรวมการทำงานของระบบ

ภาพรวมขั้นตอนการทำงานของระบบเว็บแอปพลิเคชันจัดการข้อมูล Master มี 4 ขั้นตอน ดังนี้

1. ดึงข้อมูลที่อยู่ลูกค้า
2. จัดการข้อมูลที่อยู่ลูกค้า
3. Transfer ข้อมูลที่จัดการเป็น Shipping Area Master
4. สร้างหน้าเว็บแอปพลิเคชันสำหรับ จัดการฐานข้อมูล Shipping Area Master

3.2 System Architecture Design



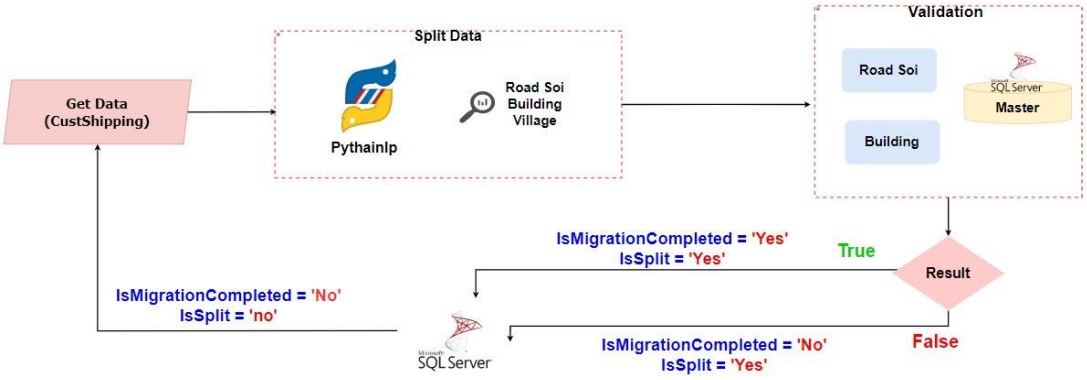
ภาพที่ 3-2 System Architecture Design

คำอธิบายภาพที่ 3-2 System Architecture Design โดยแบ่งการทำงานเป็น 2 ส่วนคือ

ส่วนที่ 1 Web Application ผู้ใช้งานสามารถเข้ามาเรียกดูข้อมูล แก้ไขข้อมูล เพิ่มข้อมูล

ส่วนที่ 2 Service ที่ไว้สำหรับจัดการข้อมูลที่อยู่ลูกค้าให้มาเป็น Shipping Area Master

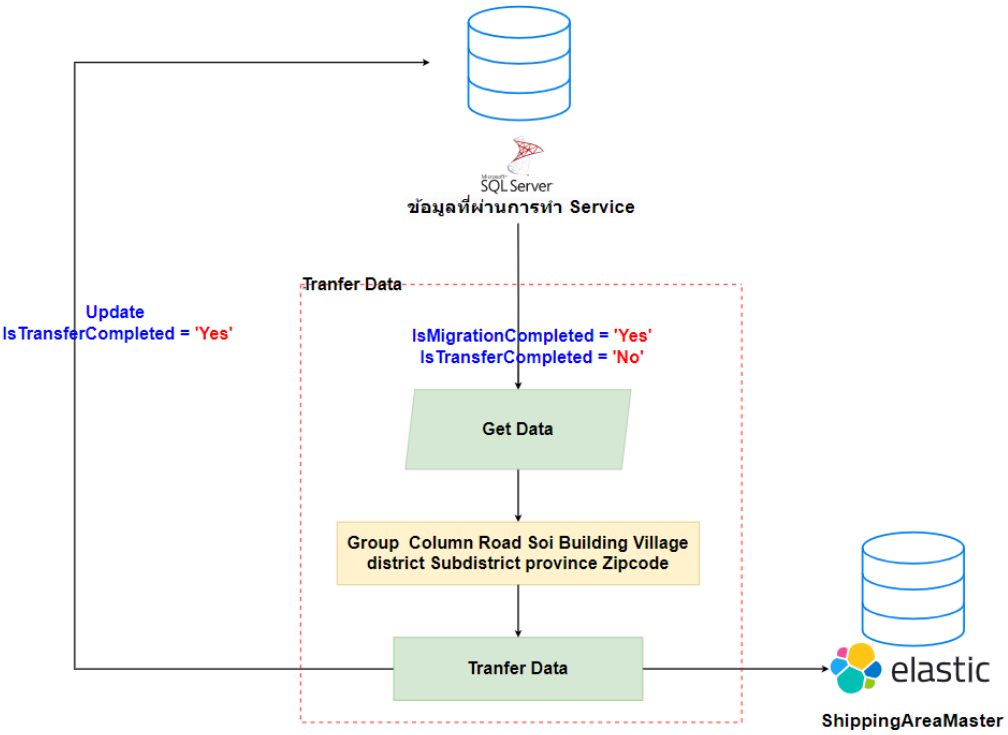
3.2.1 กระบวนการ Cleansing Data



ภาพที่ 3-3 กระบวนการ Cleansing Data

จากภาพ 3-3 service ในกระบวนการ Cleansing data มาทำการหา Road, Soi, Building, Village ให้ละเอียดขึ้นเนื่องจากตอนแรกฐานข้อมูลเก็บอยู่ในระดับที่ไม่มี Road, Soi, Building, Village เริ่มจากไปดึงข้อมูลที่อยู่อยู่ลูกค้าจากฐานข้อมูลมาทำการตัดแบ่งคำโดยใช้ Pythainlp เป็นตัวตัดแบ่งคำ หลังจากตัดแบ่งคำ จะได้ข้อมูล Road, Soi, Building, Village จากนั้นนำ ไปตรวจสอบว่าข้อมูลที่ได้มามีความถูกต้องตรงกับ Master ข้อมูลจะทำการ Transfer ได้ แต่ถ้าตรวจสอบแล้วข้อมูลมีความไม่ถูกต้องการ Transfer ก็จะไม่ทำ

3.2.2 กระบวนการ Migration

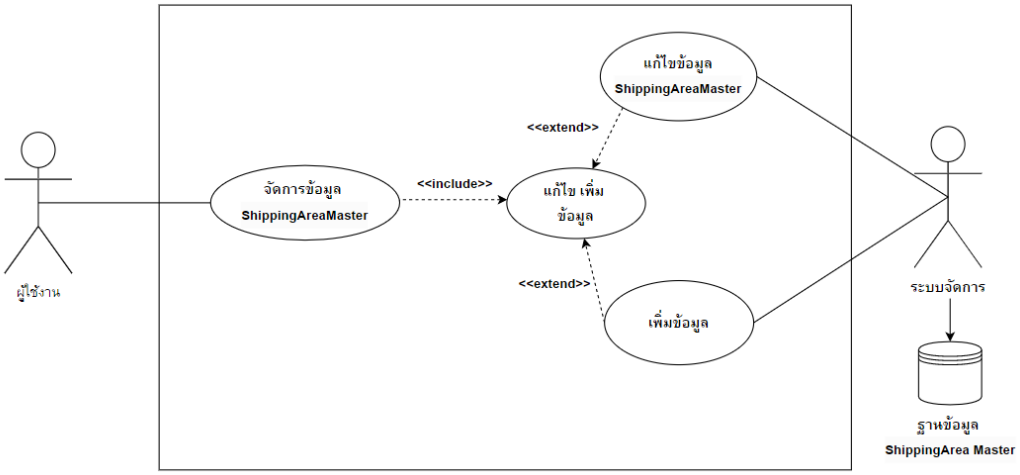


ภาพที่ 3-4 กระบวนการ Migration

จากภาพ 3-4 หลังได้ข้อมูลที่สามารถ Transfer ได้ในข้อ 3.2.1 จะนำข้อมูลที่ได้มาทำการจัดกลุ่มข้อมูลที่เหมือนกัน แล้ว Transfer ไปที่ฐานข้อมูล Elastic search เป็นข้อมูล Shipping Area Master

3.3 การจัดการข้อมูล

3.3.1 Use Case Diagram



ภาพที่ 3-5 Use Case Diagram การจัดการข้อมูล

จากภาพที่ 3-5 อธิบายได้ดังต่อไปนี้

1. ผู้ใช้งานสามารถเพิ่ม ข้อมูล Shipping Area Master ได้
2. ผู้ใช้งานสามารถแก้ไข ข้อมูล Shipping Area Master ได้

3.3.1.1 Use Case Description

Use Case Description แสดงฟังก์ชันการทำงานของผู้ใช้งานดังตาราง 3-1

ตารางที่ 3-1 Use Case Description : จัดการข้อมูล Shipping Area Master

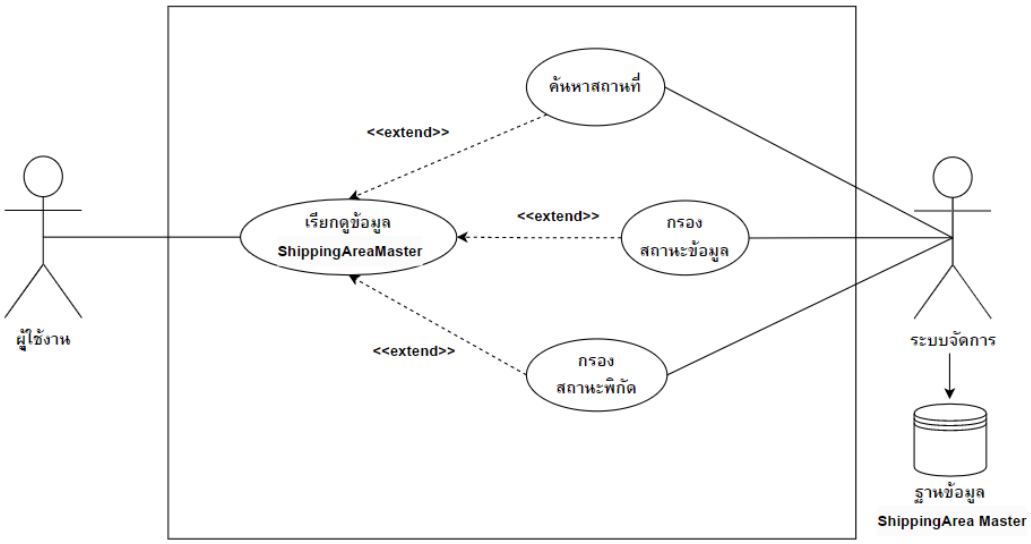
Use Case Number	1
Use Case Name	จัดการข้อมูล Location Master
Brief Description	ผู้ใช้งานสามารถ เพิ่ม แก้ไข ข้อมูล Shipping Area Master
Actors	ผู้ใช้งาน
Pre-Condition	-
Post-Condition	ผู้ใช้งานสามารถ เพิ่ม แก้ไข ข้อมูล Shipping Area Master

ตารางที่ 3-1 (ต่อ) Use Case Description : จัดการข้อมูล Shipping Area Master

Main-flow	Actor	System
	1. ผู้ใช้งานเพิ่มข้อมูล Shipping Area Master 2. ผู้ใช้งานแก้ไขข้อมูล Shipping Area Master	
Alternate/Exceptional Flows :	ถ้าผู้ใช้งานกรอกข้อมูลไม่ครบหรือกรอกข้อมูลซ้ำที่มีอยู่แล้วจะไม่สามารถเพิ่ม แก้ไข ข้อมูลลงฐานข้อมูลได้	

3.4 การดูข้อมูล

3.4.1 Use case diagram



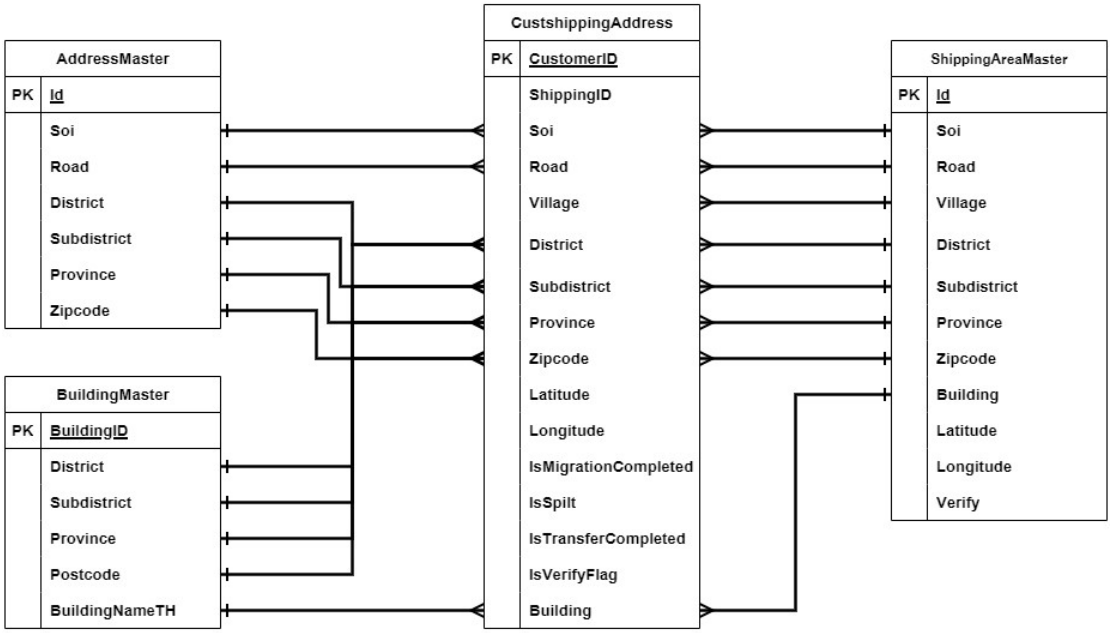
ภาพที่ 3-6 Use Case Diagram การดูข้อมูล

3.4.1.1 Use Case Description การดูข้อมูล

ตารางที่ 3-2 Use Case Description : การดูข้อมูล

Use Case Number	2	
Use Case Name	ดูข้อมูล	
Brief Description	-	
Actors	ผู้ใช้งาน	
Pre-Condition	-	
Post-Condition	ผู้ใช้งานสามารถดูข้อมูล	
Main-flow	**Actor**	**System**
	2. ผู้ใช้งานกรอกสถานที่ที่ต้องการดู เลือกสถานะข้อมูล และเลือกสถานะพิกัดที่ตั้งที่ต้องการดู	1. แสดง text search ให้ผู้ใช้งานกรอกสถานที่ที่ต้องการดู มี checkbox ให้เลือกสถานะของข้อมูล และแสดง dropdown ให้เลือกสถานะพิกัดที่ตั้งที่ต้องการดู 3. แสดงข้อมูลที่ผู้ใช้งานกรอกสถานที่มา และที่เลือกสถานะมา
Alternate/Exceptional Flows :	1.ถ้าผู้ใช้งานไม่เลือกเงื่อนไขในการค้นหาอย่างใดอย่างหนึ่งจะไม่สามารถแสดงข้อมูลให้ดูได้	

3.5 ER Diagram



ภาพที่ 3-7 ER Diagram

3.6 Data Table

ตารางที่ 3-3 แสดงตาราง BuildingMaster เป็นตารางเก็บข้อมูลตึก

Attribute	Data Type (Size)	Description	Key type
BuildingID	Int	รหัสตึก	Primary Key
District	Varchar(50)	เขตของตึก	Foreign Key (CustShippingAddress)
Subdistrict	Varchar(50)	แขวงของตึก	Foreign Key (CustShippingAddress)
Province	Varchar(50)	จังหวัดของตึก	Foreign Key (CustShippingAddress)
Zipcode	Varchar(5)	รหัสไปรษณีย์ตึก	Foreign Key (CustShippingAddress)
BuildingNameth	Varchar(100)	ชื่อของตึก	Foreign Key (CustShippingAddress)

ตารางที่ 3-4 แสดงตาราง AddressMaster ที่เก็บข้อมูลที่อยู่ของซอยถนนที่เอาไว้ตรวจสอบ

Attribute	Data Type (Size)	Description	Key type
Id	Int	รหัสสถานที่	Primary Key
Soi	Varchar(100)	ชื่อซอยของสถานที่	Foreign Key (CustShippingAddress)
Road	Varchar(100)	ชื่อถนนของสถานที่	Foreign Key (CustShippingAddress)
District	Varchar(50)	เขตของสถานที่	Foreign Key (CustShippingAddress)
Subdistrict	Varchar(50)	แขวงของสถานที่	Foreign Key (CustShippingAddress)
Province	Varchar(50)	จังหวัดของสถานที่	Foreign Key (CustShippingAddress)
Zipcode	Varchar(5)	รหัสไปรษณีย์สถานที่	Foreign Key (CustShippingAddress)

ตารางที่ 3-5 แสดงตาราง CustShippingAddress เป็นตารางเก็บข้อมูลที่อยู่ลูกค้า

Attribute	Data Type (Size)	Description	Key type
CustomerID	Int	รหัสลูกค้า	Primary Key
ShippingID	Int	รหัสที่อยู่ลูกค้า	
Soi	Varchar(100)	ชื่อซอยที่อยู่ลูกค้า	
Road	Varchar(100)	ชื่อถนนที่อยู่ลูกค้า	
Village	Varchar(100)	หมู่บ้านที่อยู่ลูกค้า	
District	Varchar(100)	เขตของที่อยู่ลูกค้า	
Subdistrict	Varchar(100)	แขวงของที่อยู่ลูกค้า	
Province	Varchar(100)	จังหวัดของที่อยู่ลูกค้า	
Zipcode	Varchar(5)	รหัสไปรษณีย์ที่อยู่ลูกค้า	
Latitude	Decimal(11,8)	ละติจูดที่อยู่ลูกค้า	
Longtitude	Decimal(11,8)	ลองจิจูดที่อยู่ลูกค้า	

ตารางที่ 3-5 (ต่อ) แสดงตาราง CustShippingAddress เป็นตารางเก็บข้อมูลที่อยู่ลูกค้า

IsMigrationCompleted	Varchar(3)	สถานะการจัดการข้อมูลที่อยู่ลูกค้า	
IsSpilt	Varchar(3)	สถานการณ์ตัดแบ่งที่อยู่ลูกค้า	
IsTransferCompleted	Varchar(3)	สถานะการโอนย้ายที่อยู่ลูกค้า	
IsVerfifyFlag	Varchar(3)	สถานะการตรวจสอบที่อยู่ลูกค้า	
Building	Varchar(100)	ตึกที่อยู่ลูกค้า	

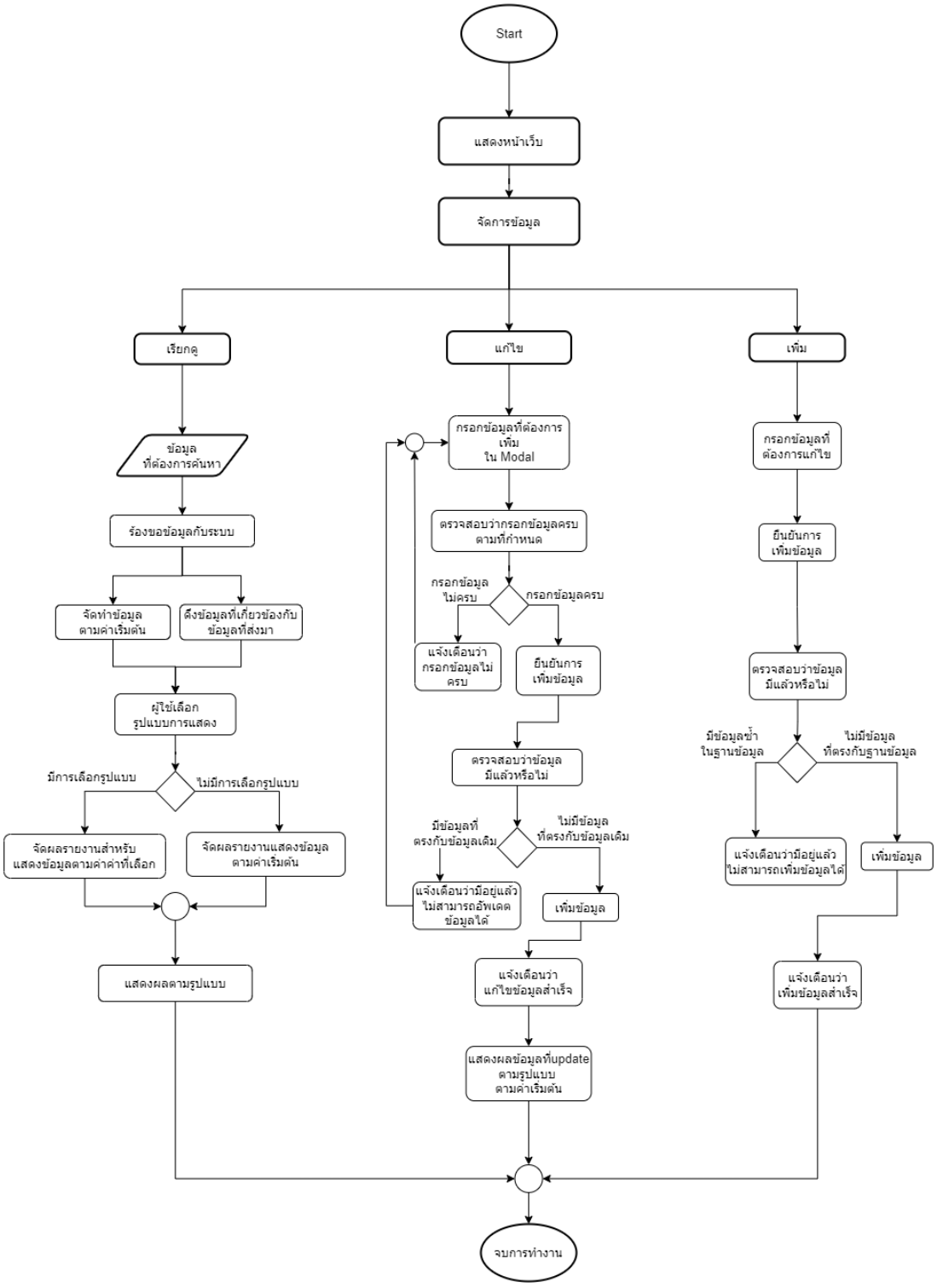
ตารางที่ 3-6 แสดงตาราง ShippingAreaMaster เป็นตารางเก็บข้อมูล Master สถานที่

Attribute	Data Type (Size)	Description	Key type
Id	Int	รหัสMaster	Primary Key
Soi	Varchar(100)	ชื่อซอยที่อยู่Master	Foreign Key (CustShippingAddress)
Road	Varchar(100)	ชื่อถนนที่อยู่Master	Foreign Key (CustShippingAddress)
Village	Varchar(100)	หมู่บ้านที่อยู่Master	Foreign Key (CustShippingAddress)
Building	Varchar(100)	ตึกที่อยู่Master	Foreign Key (CustShippingAddress)
District	Varchar(100)	เขตของที่อยู่Master	Foreign Key (CustShippingAddress)
Subdistrict	Varchar(100)	แขวงของที่อยู่Master	Foreign Key (CustShippingAddress)
Province	Varchar(100)	จังหวัดของที่อยู่Master	Foreign Key (CustShippingAddress)
Zipcode	Varchar(5)	รหัสไปรษณีย์ที่อยู่Master	Foreign Key (CustShippingAddress)

ตารางที่ 3-6 (ต่อ) แสดงตาราง ShippingAreaMaster เป็นตารางเก็บข้อมูล Master สถานที่

Latitude	Decimal(11,8)	ละติจูดที่อยู่Master	
Longtitude	Decimal(11,8)	ลองจิจูดที่อยู่Master	
Verfify	Varchar(3)	สถานะการตรวจสอบที่อยู่Master	

3.7 Flowchart



ภาพที่ 3-8 Flowchart

บทที่ 4
การพัฒนาระบบ

จากการศึกษา ค้นคว้าข้อมูลและออกแบบการทำงานและข้อมูลในระบบผู้จัดทำจึงเลือกใช้เครื่องมือและอุปกรณ์ที่ เหมาะสมในการพัฒนาระบบ รวมทั้งขั้นตอนการทำงานต่าง ๆ ที่สำคัญภายในระบบ เพื่อใช้ทดสอบระบบและพัฒนาระบบต่อไป โดยสามารถแบ่งออกได้ดั้งนี้

4.1 เครื่องมือที่ใช้ในการพัฒนาระบบ

4.1.1 เครื่องมือ

- Visual Studio 2022
- Visual Studio Code
- ภาษา C#
- ภาษา HTML
- ภาษา CSS
- Python
- Bootstrap Framework
- axios v0.20.0
- PostgREST
- Vue v2.6.10
- Microsoft SQL Server

4.1.2 ระบบปฏิบัติการที่ในการพัฒนาระบบ

- ระบบปฏิบัติการ window 11

4.2 Service การจัดการข้อมูลที่อยู่ลูกค้า

4.2.1 กระบวนการ Cleansing Data

4.2.1.1 การติดต่อฐานข้อมูล



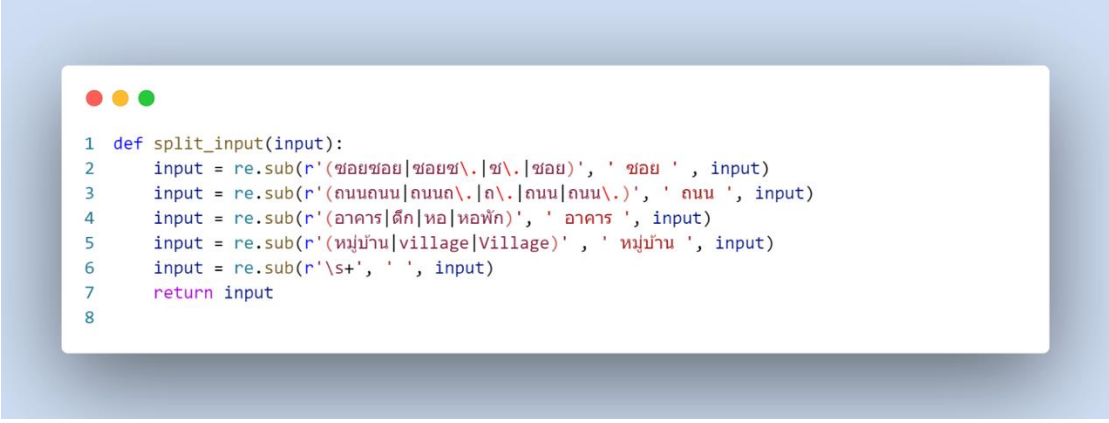
```python
def connectDB():
    # Setup Database Information
    server = config["SQLServerConnection"]["Server"]
    database = config["SQLServerConnection"]["Database"]
    username = config["SQLServerConnection"]["Username"]
    password = config["SQLServerConnection"]["Password"]
    driver = config["SQLServerConnection"]["Driver"]
    multiSubnetFailover = config["SQLServerConnection"]["MultiSubnetFailover"]
    conn = pyodbc.connect('DRIVER='+driver+';SERVER='+server +
                          ';PORT=xxxx;DATABASE='+database+';UID='+username+';PWD=' +
password+';MultiSubnetFailover=' + multiSubnetFailover)
    return conn

```

ภาพที่ 4-1 การติดต่อฐานข้อมูล

จากภาพ 4-1 เป็นการติดต่อฐานข้อมูล เพื่อให้ service สามารถ อ่าน แก้ไข ลบ ข้อมูลที่ฐานข้อมูลได้

4.2.1.2 ฟังก์ชัน spilt_input



```python
def split_input(input):
    input = re.sub(r'(ซอยซอย|ซอยซ\.|ซ\.|ซอย)', ' ซอย ' , input)
    input = re.sub(r'(ถนนถนน|ถนนถ\.|ถ\.|ถนน|ถนน\.)', ' ถนน ', input)
    input = re.sub(r'(อาคาร|ตึก|หอ|หอพัก)', ' อาคาร ', input)
    input = re.sub(r'(หมู่บ้าน|village|Village)' , ' หมู่บ้าน ', input)
    input = re.sub(r'\s+', ' ', input)
    return input

```

ภาพที่ 4-2 ฟังก์ชัน spilt_input

จากภาพ 4-2 ทำการแทนที่คำที่มีอยู่ใน input เป็นคำที่ต้องการแทนที่ อีกทั้งแทนที่ค่าช่องว่างที่ซ้ำกัน และสุดท้ายลบช่องว่าง

4.2.1.3 กระบวนการดึงข้อมูล



```python
def getCustShippingMasters(conn):
    try:
        cursor = conn.cursor()
        querySql = f'''
                SELECT
                 CustomerID as customerid
                ,ShippingID as shippingid
                ,District as district
                ,SubDistrict as subdistrict
                ,Province as province
                ,Zipcode as zipcode
                ,CONVERT(varchar(20), Latitude) as latitude
                ,CONVERT(varchar(20), Longitude) as longitude
                ,ShippingAddress as address
            FROM TBCustShippingMaster_Migration
            WHERE IsMigrationCompleted = 'No'  and  IsSpilt ='No'
            '''
        cursor.execute(querySql)
        columns = [column[0] for column in cursor.description]
        results = []
        for row in cursor.fetchall():
            rowData = dict(zip(columns, row))
            json_data = json.dumps(rowData)
            data = json.loads(json_data)
            results.append(ShippingAddress(**data))
        return results

    except Exception as ex:
```

ภาพที่ 4-3 กระบวนการดึงข้อมูล

จากภาพ 4-3 เขียน query ดึงข้อมูลที่ต้องการ

4.2.1.4 ฟังก์ชัน findRoadandSoi



```
def findRoadAndSoi(input):
    try:
        road = ''
        soi = ''
        building = ''
        building = ''
        village = ''
        keySplit = ''
        if "แขวง" in input:
            keySplit = "แขวง"
        elif "ตำบล" in input:
            keySplit = "ตำบล"
        elif "ต." in input:
            keySplit = "ต."
        if (keySplit == ''):
            strSplits = word_tokenize(split_input(input), keep_whitespace=False, engine="longest")#Spilt
            road = filterRoadAndSoiFromDataSplited(strSplits,  r'ถนน|ถนน|ถ\.')
            soi = filterRoadAndSoiFromDataSplited(strSplits, r'ซอย|ซ\.')
            building = filterRoadAndSoiFromDataSplited(strSplits, r'อาคาร')
            village = filterRoadAndSoiFromDataSplited(strSplits, r'หมู่บ้าน')
        else:
            strSplit = ''
            strSplits = input.split(keySplit)
            if strSplits and len(strSplits) > 1:
                strSplit = strSplits[0]
            else:
                strSplit = input
            strSplits = word_tokenize(split_input(strSplit), keep_whitespace=False, engine="longest")
            road = filterRoadAndSoiFromDataSplited(strSplits, r'ถนน|ถ\.')
            soi = filterRoadAndSoiFromDataSplited(strSplits,  r'ซอย|ซ\.')
            building = filterRoadAndSoiFromDataSplited(strSplits, r'อาคาร')
            village = filterRoadAndSoiFromDataSplited(strSplits, r'หมู่บ้าน')
        return road, soi, building, village
    except Exception as ex:
```

ภาพที่ 4-4 ฟังก์ชัน findRoadandSoi

จากภาพ 4-4 นำ input ที่ผ่านการทำข้อ 4.2.1.3 มาตัดแบ่ง input โดย keySpilt ที่เอาไว้แบ่ง คือ แขวง, ตำบล หลังจากตัดแบ่งจะได้ input ที่ไม่ข้อมูลตั้งแต่หลังคำ keySpilt จากนั้นนำ input มาทำการตัดแบ่งคำโดยใช้ engine “longest”, keep_whitespace “false” นำคำที่ตัดแบ่งได้ส่งไปทำฟังก์ชันต่อไป

4.2.1.5 กระบวนการหา Road, Soi, Building, Village



```
resultoutput = ''
keyWord =''
len_strSplits = len(strSplits)
filtered_strings = [s for s in strSplits if re.search(key, s)]
if filtered_strings and len(filtered_strings) > 0:
        filtered_string = filtered_strings[0]
        index = strSplits.index(filtered_string)
        next_index = index + 1
        if(next_index < len_strSplits):
            keyWord = strSplits[next_index]
            while next_index < (len_strSplits - 1):
                next_index += 1
                nextkeyWord= strSplits[next_index]
                if (nextkeyWord!= ''and nextkeyWord!= '(' and nextkeyWord!= ')' and nextkeyWord!= 'ถนน' and nextkeyWord!= key
                                    and nextkeyWord!= 'ซอย' and nextkeyWord!='อาคาร' and nextkeyWord!='หมู่บ้าน' and nextkeyWord!='หมู่' and nextkeyWord!='ม.'
                                    and nextkeyWord!='เขต' and nextkeyWord!='แยก' and nextkeyWord!='ชั้น' and nextkeyWord!='ห้อง'):
                    keyWord += nextkeyWord
                else:
                    break
            if ')' in strSplits[next_index] :
                keyWord = ''
            if keyWord.endswith('-'):
                keyWord = keyWord[:-1]

resultoutput = replace_special_charactersstart(keyWord, '')
print('keyWord',resultoutput)
return  resultoutput.replace(" ", "")
```

ภาพที่ 4-5 กระบวนการหา Road, Soi, Building, Village

จากภาพ 4-5 นำเอา array ที่ผ่านการจัดแบ่งคำมาหา Road, Soi, Building, Village โดย เอา key ที่ส่งมาเป็นตัวกำหนดว่าจะหาอะไร เพื่อทำการสลัดคำที่เกี่ยวข้องกับ key และเก็บไว้ในตัวแปร resultoutput หลังจากนั้นนำไป replace อักขระพิเศษ แล้วก็ส่งค่า resultoutput กลับไปต้นทาง โดย replace ช่องว่างออกด้วย

4.2.1.6 การ Validation Road Soi



```
def checkRoadSoiMasters(Road,Soi, district,subdistrict,province,zipcode):
    if(Soi is  None or Soi == "") and (Road is  None or Road == ""):
            return True
    conn = connectDB()
    cursor = conn.cursor()
    try:
            querySql = f'''
                  SELECT
                       Soi,
                       Road,
                       Province,
                       District,
                       Subdistrict,
                       Postcode
                   FROM
                       TBAddressMaster
                   WHERE
                        Road ='{Road}' and Soi ='{Soi}' and District = '{district}' and Province ='{province}' and Subdistrict = '{subdistrict}' and Postcode = '{zipcode}'
                   '''
            cursor.execute(querySql)
            if  len(cursor.fetchall()) > 0:
                return True
            else:
                 return False

    except Exception as ex:
```

ภาพที่ 4-6 การ Validation Road Soi

จากภาพ 4-6 หลังจากทำข้อ 4.2.1.5 จะได้ค่า Road, Soi โดยในกระบวนนี้จะทำการตรวจสอบค่า Road, Soi ว่าถูกต้องตรงกับข้อมูลในตาราง TBAddressMaster หรือไม่

4.2.2 กระบวนการ Migration

4.2.2.1 การดึงข้อมูลมาจัดกลุ่มข้อมูล



```
def getElasticsearchCustShippingMasters(conn):
    try:
        querySql = '''
        SELECT

            building,
            soi,
            road,
            village,
            district,
            subdistrict,
            province,
            zipcode,
            CONVERT(varchar(20), MAX(latitude)) AS latitude,
            CONVERT(varchar(20), MAX(longitude)) AS longitude
        FROM TBCustShippingMaster_Migration
        WHERE IsMigrationCompleted = 'No' AND IsSpilt ='Yes'
        AND IsTransferCompleted = 'No' AND  district <>''  AND  subdistrict <>'' AND  province <>'' AND  zipcode <>''
        GROUP BY building, soi, road, village, district, subdistrict, province, zipcode
        '''
        with conn.cursor() as cursor:
            cursor.execute(querySql)
            columns = [column[0] for column in cursor.description]
            results = [ElasticsearchShippingAddress(
                **dict(zip(columns, row))) for row in cursor.fetchall()]
            return results
    except Exception as ex:
```

ภาพที่ 4-7 การดึงข้อมูลมาจัดกลุ่มข้อมูล

จากภาพ 4-7 ทำการดึงข้อมูลผ่านการทำกระบวนการ Cleansing Data โดยสถานะ IsSpilt “Yes” และสถานะ IsMigrationcompleted “No” หมายความว่ายังไม่ถูกทำการ Migration หลังจากดึงข้อมูลมาแล้วก็ Group By building, soi, road, village, district, subdistrict, province, zipcode

4.2.2.2 การ Transfer ข้อมูล

```python
def pushBulkDataToElasticSearch(data_list):
    bulk_data = generate_bulk_data(data_list)

    url = config['ElasticOptions']['Url']
    index = config["ElasticOptions"]["CollectionName"]["CustShippingMaster"]
    headers = {'Content-Type': 'application/x-ndjson'}

    response = requests.post(f"{url}{index}/_bulk", data=bulk_data, headers=headers)

    return response
```

ภาพที่ 4-8 การ Transfer ข้อมูล

จากภาพ 4-8 ทำการตั้งค่าพารามิเตอร์สำหรับการ Transfer และส่งคำขอไปยัง bulk ตำแหน่งข้อมูลของ Elastic Search ด้วยข้อมูลที่สร้างขึ้น

4.2.2.3 การเตรียมข้อมูลแบบ bulk

```python
def generate_bulk_data(custShippingMasters):
    index = config["ElasticOptions"]["CollectionName"]["CustShippingMaster"]
    bulk_data = []
    for item in custShippingMasters:
        id = str(bson.ObjectId())
        action = {
            "index": {
                "_index": index,
                "_type": "_doc",
                "_id"  : id
            }
        }
        data = generate_data(id,item)
        bulk_data.append(json.dumps(action))
        bulk_data.append(json.dumps(data))
    return "\n".join(bulk_data) + "\n"
```

ภาพที่ 4-9 การเตรียมข้อมูลแบบ bulk

จากภาพ 4-9 ทำหน้าที่จัดรูปแบบข้อมูลในรูปแบบที่เหมาะกับการทำ index แบบ bulk ใน Elasticsearch และจากนั้นส่งข้อมูลไปที่ index ที่ระบุใน Elasticsearch โดยใช้ API endpoint “bulk” ข้อมูลถูกจัดรูปแบบเป็น string JSON ที่ขึ้นบรรทัดใหม่ ซึ่งเป็นรูปแบบที่ Elasticsearch ต้องการ

4.2.2.4 การสร้างเนื้อหาของข้อมูล



```
def generate_data(id,row):
    document_body = {
            "id" :id,
            "soi": row.soi,
            "road": row.road,
            "building": row.building,
            "village": row.village,
            "subdistrict": row.subdistrict,
            "district": row.district,
            "province": row.province,
            "zipcode": row.zipcode,
            "latitude": float(row.latitude),
            "longitude": float(row.longitude),
            "verify": 'No'
        }
    return   document_body
```

ภาพที่ 4-10 การสร้างเนื้อหาของข้อมูล

จากภาพ 4-10 ฟังก์ชันนี้ทำหน้าที่สร้างเนื้อหาข้อมูลของเอกสารที่จะถูกนำเข้า Elasticsearch โดยมีโครงสร้าง และข้อมูลที่ได้รับมาจาก row object และ id ที่ถูกสร้างขึ้น

4.3 การพัฒนาระบบส่วนการแสดงผล

4.3.1 การติดต่อ Service Google Map API



```
const googleMapApi = await GoogleMapsApiLoader({ apiKey: this.apiKey })
      this.loader = new Loader({
        apiKey: this.apiKey,
        version: 'weekly',
        libraries: ['places'],
      })

      this.google = googleMapApi
      var url = ''
```

ภาพที่ 4-11 การติดต่อ Service Google Map API

จากภาพ 4-11 เป็นการตั้งค่าและโหลด Google Map API ด้วยคีย์ API และระบุ libraries ต้องการดู

4.3.1.1 การสร้างแผนที่



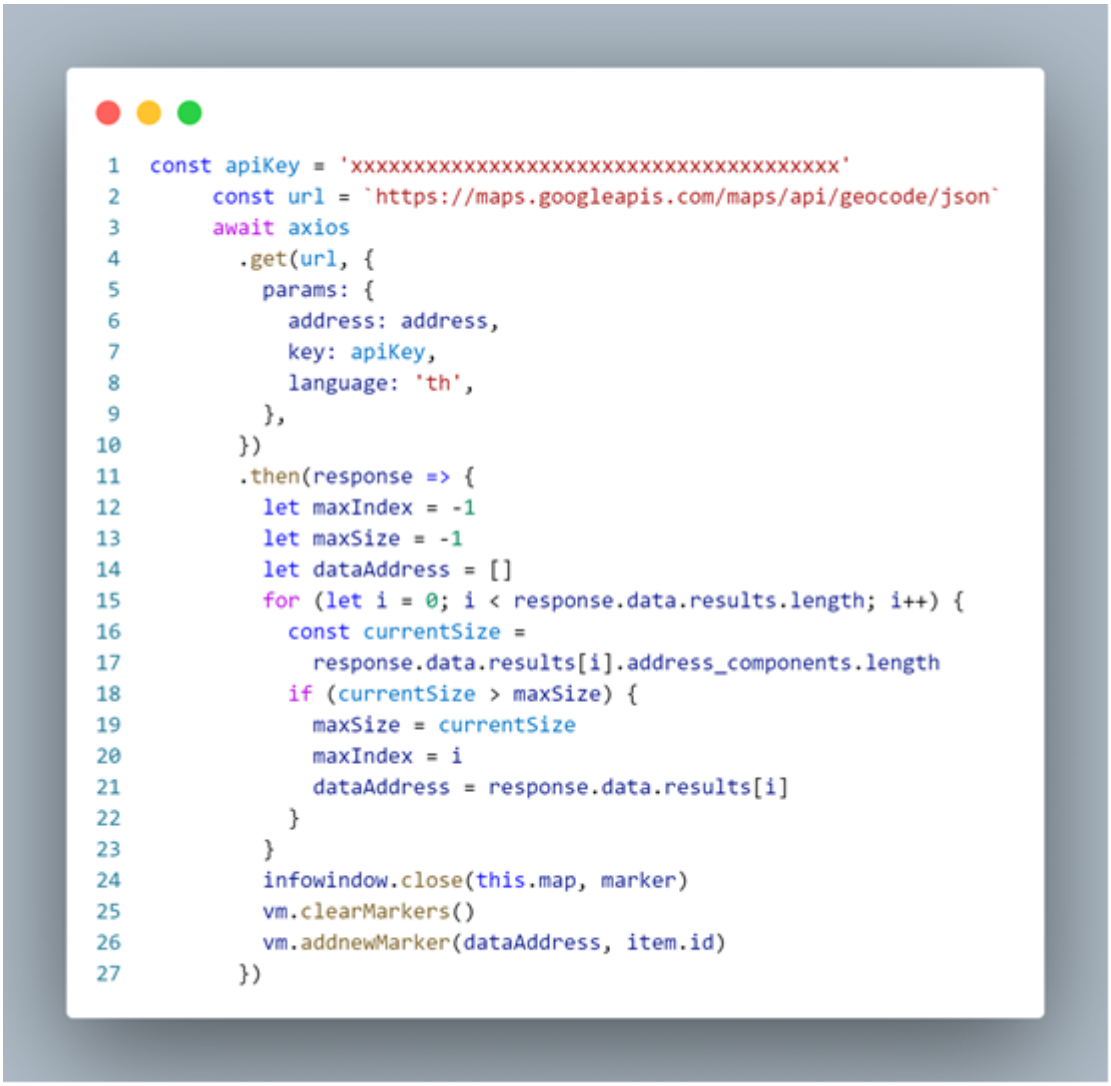
```
this.map = new googleMapApi.maps.Map(document.getElementById(mapId), {
        zoom: 11,
        center: { lat: 13.756738, lng: 100.501874 },
        minZoom: 6,
        maxZoom: 18,
        zoomControl: false,
        mapTypeControl: false,
        scaleControl: false,
        streetViewControl: false,
        rotateControl: false,
      })

      var ctaLayer = new googleMapApi.maps.KmlLayer({
        url: url,
        map: this.map,
      })
```

ภาพที่ 4-12 การสร้างแผนที่

จากภาพ 4-12 เป็นการสร้างแผนที่ Google Map และเพิ่ม KML Layer เพื่อแสดงข้อมูลที่ระบุใน KML ลงบนแผนที่นั้น และเพื่อเตรียมไว้เรียกใช้งานต่อไป

4.3.2 การร้องขอข้อมูลเกี่ยวกับพิกัดทางภูมิศาสตร์

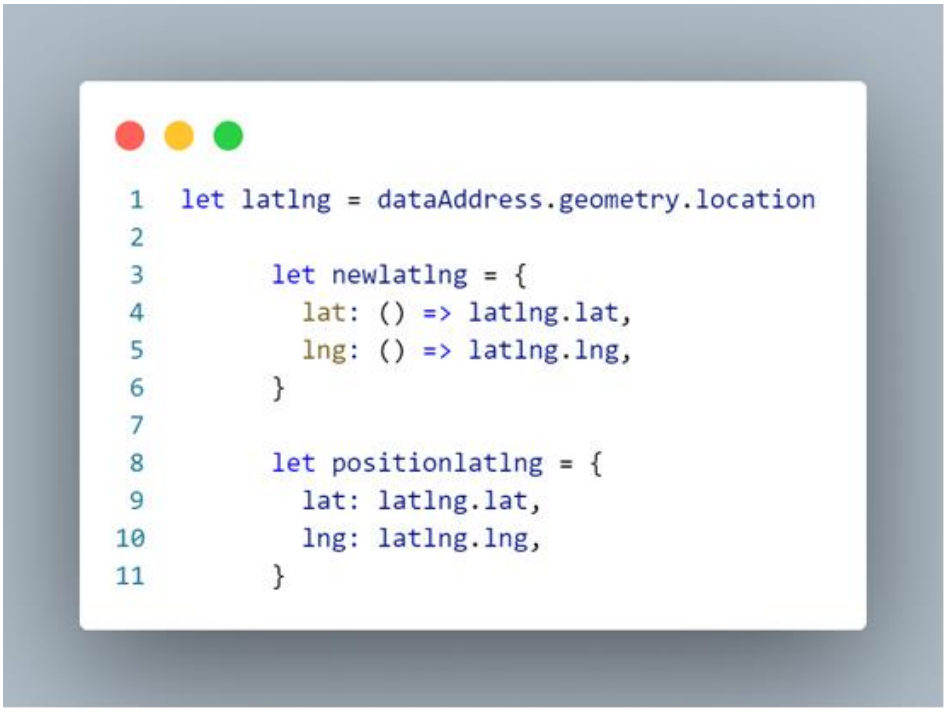


```
const apiKey = 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'
    const url = `https://maps.googleapis.com/maps/api/geocode/json`
    await axios
      .get(url, {
        params: {
          address: address,
          key: apiKey,
          language: 'th',
        },
      })
      .then(response => {
        let maxIndex = -1
        let maxSize = -1
        let dataAddress = []
        for (let i = 0; i < response.data.results.length; i++) {
          const currentSize =
            response.data.results[i].address_components.length
          if (currentSize > maxSize) {
            maxSize = currentSize
            maxIndex = i
            dataAddress = response.data.results[i]
          }
        }
        infowindow.close(this.map, marker)
        vm.clearMarkers()
        vm.addnewMarker(dataAddress, item.id)
      })
```

ภาพที่ 4-13 การร้องขอข้อมูลเกี่ยวกับพิกัด

จากภาพ 4-13 เป็นการขอข้อมูลตำแหน่งทางภูมิศาสตร์ (ละติจูด ลองจิจูด) โดยขอจาก Geocoding API ตามที่อยู่ที่ให้ไว้ จากนั้นประมวลผลการตอบสนองเพื่อค้นหาผลลัพธ์ และให้ส่งข้อมูลกลับไปใช้

4.3.2.1 การใช้ข้อมูลจาก Geocoding API



```
let latlng = dataAddress.geometry.location

      let newlatlng = {
        lat: () => latlng.lat,
        lng: () => latlng.lng,
      }

      let positionlatlng = {
        lat: latlng.lat,
        lng: latlng.lng,
      }
```

ภาพที่ 4-14 การใช้ข้อมูลจาก Geocoding API

จากภาพ 4-14 เป็นการดึงค่า dataAddress ที่ได้มาจาก 4.3.2 โดยต้องการข้อมูลตำแหน่งทางภูมิศาสตร์ (ละติจูด ลองจิจูด) โดยจะได้มาจาก geometry.location ของ dataAddress

4.3.2.2 การร้องขอที่อยู่ Address จากพิกัดทางภูมิศาสตร์



```
async getFormatAddress(LatLng) {
      return new Promise((resolve, reject) => {
        const geocoder = new this.google.maps.Geocoder()
        const geocoderRequest = {
          location: LatLng,
          language: 'th',
        }
        geocoder.geocode(geocoderRequest, (results, status) => {
          if (status === 'OK') {
            let maxSize = -1
            let formatAddress = ''

            for (let i = 0; i < results.length; i++) {
              const currentSize = results[i].address_components.length
              if (currentSize > maxSize) {
                maxSize = currentSize
                formatAddress = results[i].formatted_address
              }
            }
            resolve(formatAddress)
          } else {
            reject(new Error(status))
          }
        })
      })
    }
```

ภาพที่ 4-15 การร้องขอ Address จากพิกัด

จากภาพ 4-15 เป็นกระบวนการแปลงพิกัดทางภูมิศาสตร์ (ละติจูด ลองจิจูด) ให้เป็นที่อยู่ของพิกัดนั้น

4.3.2.3 การสร้าง Google Map Marker

```
for (var k = 0; k < this.ShippingAreaMaster.items.length; k++) {
          const data = this.ShippingAreaMaster.items[k]
          let latlngShippingAreaMaster = {
            lat: Number(this.ShippingAreaMaster.items[k].latitude),
            lng: Number(this.ShippingAreaMaster.items[k].longitude),
          }

          this.tempId = this.ShippingAreaMaster.items[k].id

          const marker = new this.google.maps.Marker({
            position: latlngShippingAreaMaster,
            map: this.map,
            draggable: true,
            animation: this.google.maps.Animation.DROP,
            id: this.tempId,
            item: this.ShippingAreaMaster.items[k],
          })
```

ภาพที่ 4-16 การสร้าง Google Map Marker

จากภาพ 4-16 เป็นการสร้าง Google Map Marker ที่ระบุบนแผนที่ จากพิกัดข้อมูล latlngShippingAreaMaster โดยเครื่องหมายถูกตั้งค่าให้สามารถลากได้ มีการเคลื่อนไหวแบบวาง

4.3.3 การร้องขอฐานข้อมูล

```
await this.$locationmasterservice.GetLocationMasterListAsync(params).then(
        result => {
          this.ShippingAreaMaster = result.data.data
          this.activateTab = true
          this.map = null
        },
        error => {
          this.items = []
        }
      )
```

ภาพที่ 4-17 การร้องขอฐานข้อมูล

จากภาพ 4-17 เป็นการติดต่อ API เพื่อขอข้อมูล ถ้ามีค่าส่งกลับมาจะเก็บไว้ในตัวแปร Shipping Area Master ถ้าไม่มีค่าส่งกลับมา item จะเป็นค่าว่าง

4.3.3.1 การกำหนดค่า item



```
ShippingAreaMaster: function(value) {
  this.totalPages = value.totalPages
  this.firstRowOnPage = (value.pageNo - 1) * value.pageSize + 1
  this.lastRowOnPage =
    value.pageSize <= 0
      ? value.totalRecords
      : Math.min(value.pageNo * value.pageSize, value.totalRecords)
  this.totalRecords = value.totalRecords
  this.items = value.items
  this.perPage = value.pageSize
  this.pageNo = value.pageNo
}
```

ภาพที่ 4-18 การกำหนดค่า item

จากภาพ 4-18 หลังจากได้ค่า Shipping Area Master จากข้อ 4.3.3 ก็คือค่า value จากนั้น เอา value เก็บไว้ในตัวแปร items เพื่อนำข้อมูลไปแสดงผลต่อไป

4.3.4 การนำข้อมูลจากฐานข้อมูลมาแสดงผล

```
<b-table
  class="table table-centered table-nowrap"
  ref="shipmentTable"
  :empty-text="'ไม่พบข้อมูล'"
  :show-empty="items.length == 0"
  :bordered="true"
  :filter="searchShippingAreaMaster"
  :items="items"
  align="center"
  v-bind:fields="fields"
  @row-clicked="onRowClick"
  @filtered="onFiltered"
  :perPage="perPage"
>
  <template v-slot:row-clicked="data">
    <tr @click="onRowClick(data.item)">
      <slot v-bind="data" />
    </tr>
  </template>
  <template v-slot:cell(id)="data">
    <span >{{modifyId(data.item.id)}}</span>
  </template>
  <template v-slot:cell(verify)="data">
    <div class="text-center">
      <label
        v-if="data.item.verify == 'Yes'"
        class="checkbox-inline checkbox-width"
      >
        <span class="label-status label-open"
          >ตรวจสอบแล้ว</span
        >
      </label>
      <label
        v-if="data.item.verify == 'No'"
        class="checkbox-inline checkbox-width"
      >
        <span class="label-status label-pending"
          >รอดำเนินการ</span
        >
      </label>
    </div>
  </template>
</b-table>
```

ภาพที่ 4-19 การนำข้อมูลจากฐานข้อมูลมาแสดงผล

จากภาพ 4-19 เป็นการร้องขอข้อมูลจากฐานข้อมูล เก็บไว้ในตัวแปร item จากนั้นนำไปแสดงผลในหน้าเว็บแอปพลิเคชันในรูปแบบตาราง

4.3.5 InfoWindow

4.3.5.1 การสร้าง InfoWindow



```
const infowindow = new this.google.maps.InfoWindow({
          maxHeight: 400,
          maxWidth: 700,
        })

        const infoWindowContent = this.showinfo_location(
          this.masterSelect,
          masterId,
          formattedAddress
        )
        infowindow.setContent(infoWindowContent)
```

ภาพที่ 4-20 การสร้าง InfoWindow

จากภาพ 4-20 ทำหน้าที่สร้าง InfoWindow ของ Google Map และกำหนดเนื้อหาให้กับ InfoWindow โดย InfoWindow นี้จะถูกใช้เพื่อแสดงข้อมูลที่ได้จากที่ตั้งบนแผนที่

4.3.5.2 การเรียกใช้ InfoWindow



```
infowindow.open(this.map, this.createMarker)
        infowindow.addListener('closeclick', () => {
          this.createMarker.setMap(null)
        })
```

ภาพที่ 4-21 การเรียกใช้ InfoWindow

จากภาพ 4-21 เป็น event การทำงานของ InfoWindow

4.4 API ที่สร้างให้ระบบเรียกใช้งาน

4.4.1 GetLocationMasterSelectAsync

ทำหน้าที่ในการดึงข้อมูล Shipping Area Master จากฐานข้อมูลตามพารามิเตอร์ keyword โดยหากทำงานสำเร็จจะส่งข้อมูลที่เกี่ยวข้องกับ keyword ไปแสดงให้ผู้ใช้ แต่หากเกิดข้อผิดพลาดในการดึงข้อมูลจะส่งข้อผิดพลาดที่เกิดขึ้นนั้นกลับไปให้ผู้ใช้แทนดังภาพที่ 4-22

```
public async Task<List<LocationmasterMasterModel>> GetLocationMasterSelectAsync(string keyword)
{
    var wildcardQueries = new List<QueryContainer>
    {
        new WildcardQuery { Field = nameof(LocationmasterMasterModel.soi), Value = "*" + keyword + "*" },
        new WildcardQuery { Field = nameof(LocationmasterMasterModel.road), Value = "*" + keyword + "*" },
        new WildcardQuery { Field = nameof(LocationmasterMasterModel.village), Value = "*" + keyword + "*" },
        new WildcardQuery { Field =  nameof(LocationmasterMasterModel.building), Value = "*" + keyword + "*" },
        new WildcardQuery { Field = nameof(LocationmasterMasterModel.district), Value = "*" + keyword + "*" },
        new WildcardQuery { Field = nameof(LocationmasterMasterModel.subdistrict), Value = "*" + keyword + "*" },
        new WildcardQuery { Field = nameof(LocationmasterMasterModel.province), Value = "*" + keyword + "*" },
        new WildcardQuery { Field = nameof(LocationmasterMasterModel.zipcode), Value = "*" + keyword + "*" }
    };
    var boolQuery = new BoolQuery
    {
        Should = wildcardQueries
    };
    var response = await elasticUnitOfWork.CustShippingRepository.SearchAsync(boolQuery);
    if (!response.Any() || response.Count() < 0)
    {
        throw new APIResponseException(HttpStatusCode.NotFound, CustomErrorCode.C102, $" ไม่พบข้อมูล");
    }
    var data = response.ToMap<List<LocationmasterMasterModel>>()
                .OrderBy(e => e.province)
                .ThenBy(e => e.district)
                .ThenBy(e => e.subdistrict)
                .ToList();
    return data;
}
```

ภาพที่ 4-22 GetLocationMasterSelectAsync

4.4.2 GetLocationMasterListAsync

ทำหน้าที่ในการดึงข้อมูล Shipping Area Master จากเงื่อนไขในการค้นหาข้อมูลจากหน้าเว็บแอปพลิเคชัน โดยหากทำงานสำเร็จจะส่งข้อมูลที่เกี่ยวข้องกับเงื่อนไขการค้นหาไปแสดงให้ผู้ใช้ แต่หากเกิดข้อผิดพลาดในการดึงข้อมูลจะส่งข้อผิดพลาดที่เกิดขึ้นนั้นกลับไปให้ผู้ใช้แทนดังภาพที่ 4-23

```
public async Task<IPagination<LocationmasterMasterModel>> GetLocationMasterListAsync(string isLatLng, List<string>
isVerify, string road, string soi, string building, string village, string district, string subdistrict, string province,
string zipcode, int page, int pageSize)
 {
     IPagination<LocationmasterMasterModel> result = new Pagination<LocationmasterMasterModel>();
     List<QueryContainer> queryContainer = new List<QueryContainer>();
     if (!String.IsNullOrEmpty(isLatLng) && isLatLng != "All")
     {
         if (isLatLng == DecisionTypeEnum.Yes.ToString())
         {
             queryContainer.Add(new NumericRangeQuery
             {
                 Field = new Field("latitude"),
                 GreaterThan = 0
             });
         }
         else
         {
             queryContainer.Add(new NumericRangeQuery
             {
                 Field = new Field("latitude"),
                 LessThanOrEqualTo = 0
             });
         }
     }
     if (isVerify.Any() && isVerify.Count() > 0)
     {
         List<QueryContainer> listQueryContainerVerify = new List<QueryContainer>();
         foreach (var item in isVerify)
         {
             listQueryContainerVerify.Add(new MatchPhraseQuery
             {
                 Field = new Field(nameof(LocationmasterMasterModel.verify)),
                 Query = item
             });
         }
         queryContainer.Add(new BoolQuery
         {
             Should = listQueryContainerVerify
         });
     }
     if (!string.IsNullOrEmpty(road))
     {
         queryContainer.Add(new MatchPhraseQuery
         {
             Field = new Field(nameof(LocationmasterMasterModel.road)),
             Query = road
         });
     }
     if (!string.IsNullOrEmpty(soi))
     {
         queryContainer.Add(new MatchPhraseQuery
         {
             Field = new Field(nameof(LocationmasterMasterModel.soi)),
             Query = soi
         });
     }
```

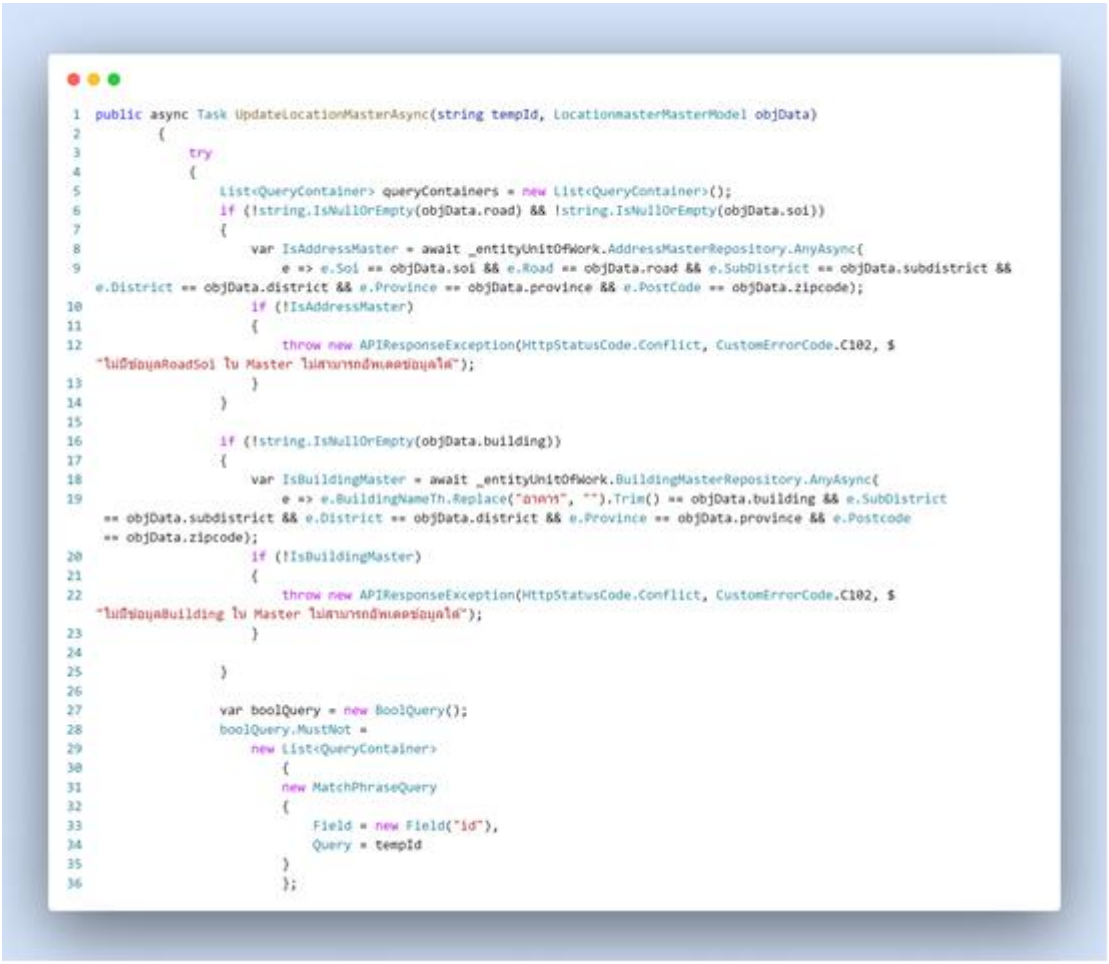
ภาพที่ 4-23 GetLocationMasterListAsync

```
        if (!string.IsNullOrEmpty(building))
        {
            queryContainer.Add(new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.building)),
                Query = building
            });
        }
        if (!string.IsNullOrEmpty(village))
        {
            queryContainer.Add(new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.village)),
                Query = village
            });
        }
        if (!string.IsNullOrEmpty(district))
        {
            queryContainer.Add(new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.district)),
                Query = district
            });
        }
        if (!string.IsNullOrEmpty(subdistrict))
        {
            queryContainer.Add(new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.subdistrict)),
                Query = subdistrict
            });
        }
        if (!string.IsNullOrEmpty(province))
        {
            queryContainer.Add(new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.province)),
                Query = province
            });
        }
        if (!string.IsNullOrEmpty(zipcode))
        {
            queryContainer.Add(new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.zipcode)),
                Query = zipcode
            });
        }
        var boolQuery = new BoolQuery
        {
            Must = queryContainer,
        };
        var response = await elasticUnitOfWork.CustShippingRepository.SearchAsync(boolQuery);
        if (!response.Any() || response.Count() < 0)
        {
            throw new APIResponseException(HttpStatusCode.NotFound, CustomErrorCode.C102, $"ไม่พบข้อมูล");
        }
        var paging = new Pagination<object>().GetPagination(response, page, pageSize);
        result = paging.ToMap<Pagination<LocationmasterMasterModel>>();
        return result;
    }
```

ภาพที่ 4-23 (ต่อ) GetLocationMasterListAsync

4.4.3 UpdateLocationMastertAsync

ทำหน้าที่ในการอัปเดตข้อมูลในฐานข้อมูล Shipping Area Master โดยก่อนการอัปเดตจะเช็คข้อมูลที่เป็น Object โดยจะเช็คข้อมูล Road Soi ก่อนว่า มีอยู่ในฐานข้อมูล AddressMaster หรือไม่ และ เช็คข้อมูล building ว่ามีอยู่ในฐานข้อมูล BuildingMaster หรือไม่ ถ้าเช็คแล้วว่า Road Soi Building มีอยู่ใน Master ต่อไปก็เช็คว่าข้อมูลทั้ง Object มีอยู่ในฐานข้อมูล Shipping Area Master หรือไม่ ถ้าเช็คแล้วไม่มีข้อมูลที่ส่งมาในฐานข้อมูล ก็จะส่งข้อมูลไปอัปเดตในฐานข้อมูล แต่ถ้ามีอยู่แล้วในฐานข้อมูลจะส่งข้อผิดพลาดที่เกิดขึ้นนั้นกลับไปให้ผู้ใช้แทน ดังภาพที่ 4-24



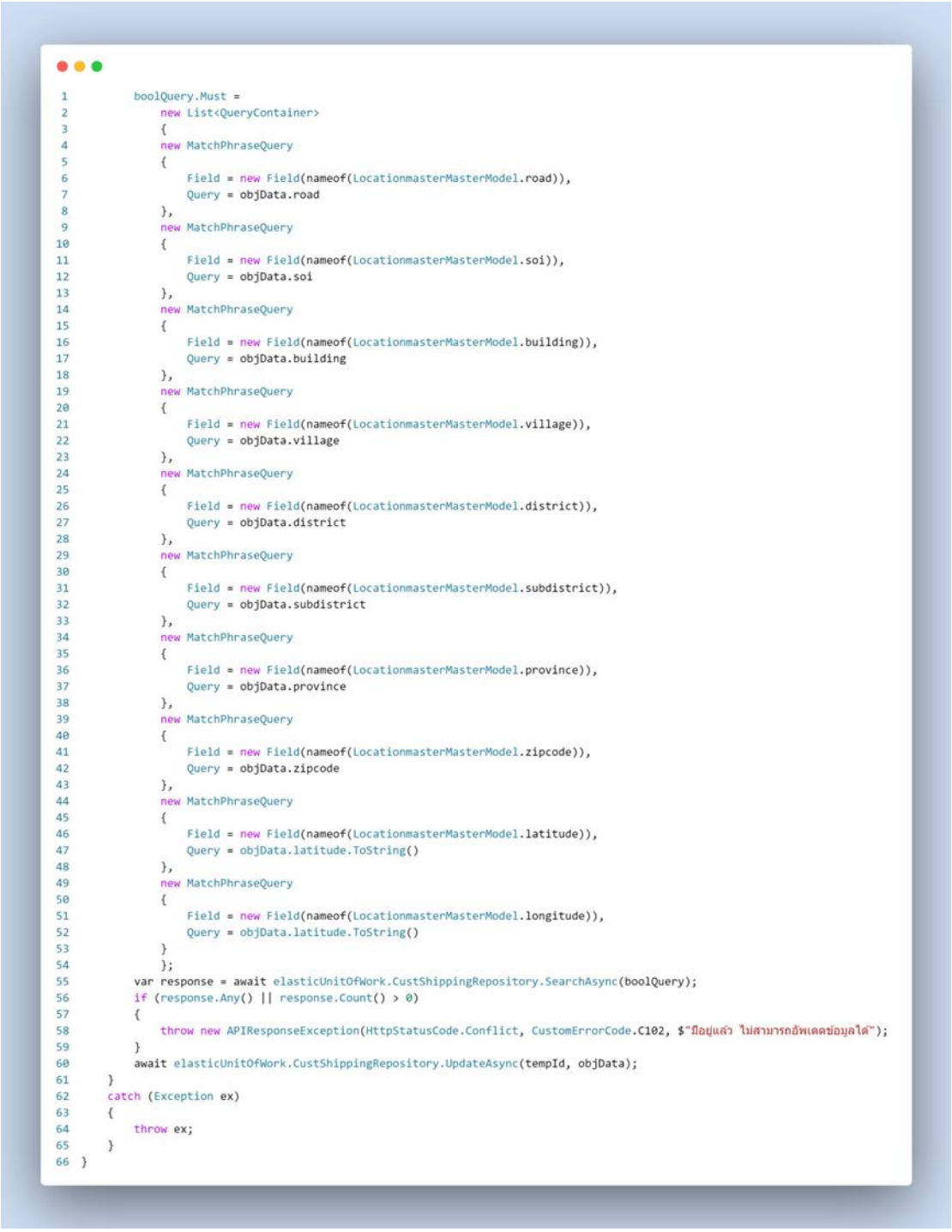
```
public async Task UpdateLocationMasterAsync(string tempId, LocationmasterMasterModel objData)
        {
            try
            {
                List<QueryContainer> queryContainers = new List<QueryContainer>();
                if (!string.IsNullOrEmpty(objData.road) && !string.IsNullOrEmpty(objData.soi))
                {
                    var IsAddressMaster = await _entityUnitOfWork.AddressMasterRepository.AnyAsync(
                        e => e.Soi == objData.soi && e.Road == objData.road && e.SubDistrict == objData.subdistrict &&
e.District == objData.district && e.Province == objData.province && e.PostCode == objData.zipcode);
                    if (!IsAddressMaster)
                    {
                        throw new APIResponseException(HttpStatusCode.Conflict, CustomErrorCode.C102, $
"ไม่มีข้อมูลRoadSoi ใน Master ไม่สามารถอัพเดตข้อมูลได้");
                    }
                }

                if (!string.IsNullOrEmpty(objData.building))
                {
                    var IsBuildingMaster = await _entityUnitOfWork.BuildingMasterRepository.AnyAsync(
                        e => e.BuildingNameTh.Replace("อาคาร", "").Trim() == objData.building && e.SubDistrict
 == objData.subdistrict && e.District == objData.district && e.Province == objData.province && e.Postcode
 == objData.zipcode);
                    if (!IsBuildingMaster)
                    {
                        throw new APIResponseException(HttpStatusCode.Conflict, CustomErrorCode.C102, $
"ไม่มีข้อมูลBuilding ใน Master ไม่สามารถอัพเดตข้อมูลได้");
                    }

                }

                var boolQuery = new BoolQuery();
                boolQuery.MustNot =
                    new List<QueryContainer>
                    {
                    new MatchPhraseQuery
                    {
                        Field = new Field("id"),
                        Query = tempId
                    }
                    };
```

ภาพที่ 4-24 UpdateLocationMastertAsync

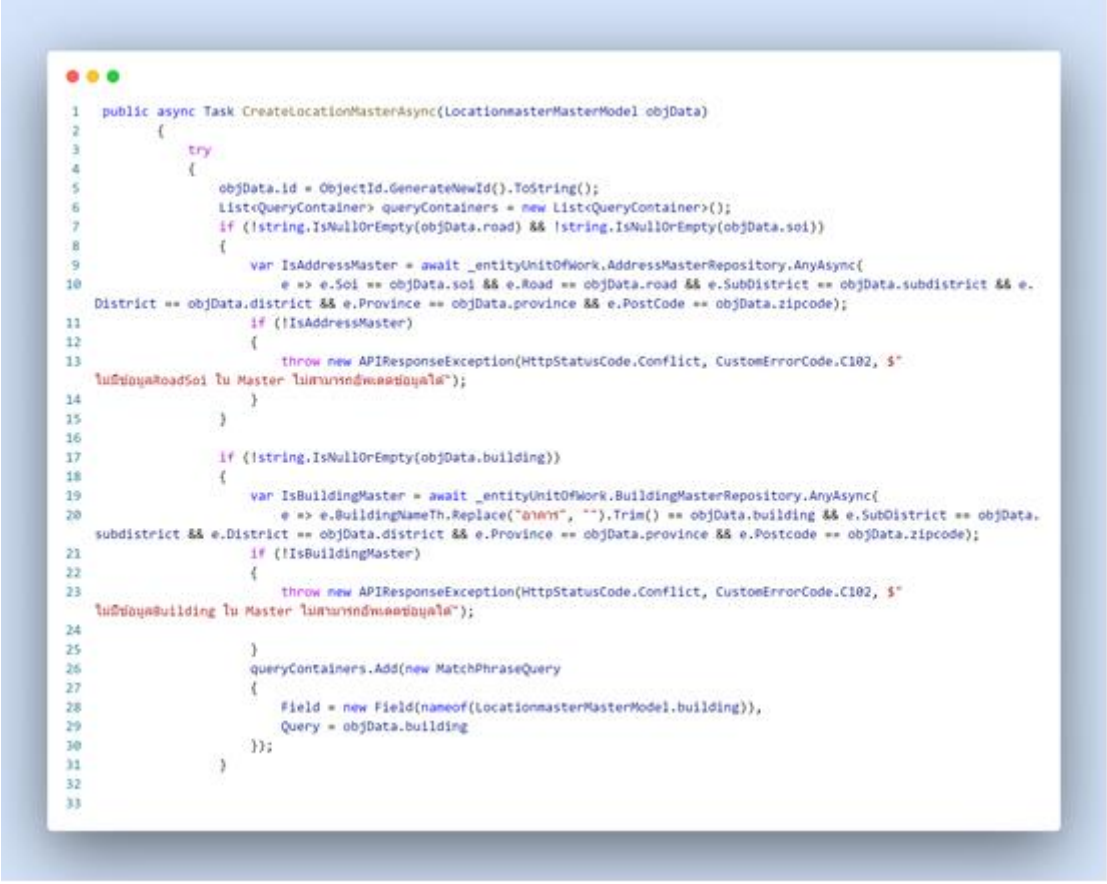


```
        boolQuery.Must =
            new List<QueryContainer>
            {
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.road)),
                Query = objData.road
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.soi)),
                Query = objData.soi
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.building)),
                Query = objData.building
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.village)),
                Query = objData.village
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.district)),
                Query = objData.district
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.subdistrict)),
                Query = objData.subdistrict
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.province)),
                Query = objData.province
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.zipcode)),
                Query = objData.zipcode
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.latitude)),
                Query = objData.latitude.ToString()
            },
            new MatchPhraseQuery
            {
                Field = new Field(nameof(LocationmasterMasterModel.longitude)),
                Query = objData.latitude.ToString()
            }
            };
        var response = await elasticUnitOfWork.CustShippingRepository.SearchAsync(boolQuery);
        if (response.Any() || response.Count() > 0)
        {
            throw new APIResponseException(HttpStatusCode.Conflict, CustomErrorCode.C102, $"มีอยู่แล้ว ไม่สามารถอัพเดตข้อมูลได้");
        }
        await elasticUnitOfWork.CustShippingRepository.UpdateAsync(tempId, objData);
    }
    catch (Exception ex)
    {
        throw ex;
    }
}
```

ภาพ 4-24 (ต่อ) UpdateLocationMastertAsync

4.4.4 CreateLocationMastertAsync

ทำหน้าที่ในการอัปเดตข้อมูลในฐานข้อมูล Shipping Area Master โดยก่อนการอัปเดตจะเช็คข้อมูลที่เป็น Object โดยจะเช็คข้อมูล Road Soi ก่อนว่า มีอยู่ในฐานข้อมูล AddressMaster หรือไม่ และเช็คข้อมูล building ว่ามีอยู่ในฐานข้อมูล BuildingMaster หรือไม่ ถ้าเช็คแล้วว่า Road Soi Building มีอยู่ใน Master ต่อไปก็เช็คว่าข้อมูลทั้ง Object มีอยู่ในฐานข้อมูล Shipping Area Master หรือไม่ ถ้าเช็คแล้วไม่มีข้อมูลที่ส่งมาในฐานข้อมูล ก็จะทำการเพิ่มข้อมูลใหม่เข้าไปในฐานข้อมูล แต่ถ้ามีอยู่แล้วในฐานข้อมูลจะส่งข้อผิดพลาดที่เกิดขึ้นนั้นกลับไปให้ผู้ใช้แทน ดังภาพที่ 4-25



```
public async Task CreateLocationMasterAsync(LocationmasterMasterModel objData)
        {
            try
            {
                objData.id = ObjectId.GenerateNewId().ToString();
                List<QueryContainer> queryContainers = new List<QueryContainer>();
                if (!string.IsNullOrEmpty(objData.road) && !string.IsNullOrEmpty(objData.soi))
                {
                    var IsAddressMaster = await _entityUnitOfWork.AddressMasterRepository.AnyAsync(
                        e => e.Soi == objData.soi && e.Road == objData.road && e.SubDistrict == objData.subdistrict && e.
District == objData.district && e.Province == objData.province && e.PostCode == objData.zipcode);
                    if (!IsAddressMaster)
                    {
                        throw new APIResponseException(HttpStatusCode.Conflict, CustomErrorCode.C102, $"
ไม่มีข้อมูลRoadSoi ใน Master ไม่สามารถอัพเดตข้อมูลได้");
                    }
                }

                if (!string.IsNullOrEmpty(objData.building))
                {
                    var IsBuildingMaster = await _entityUnitOfWork.BuildingMasterRepository.AnyAsync(
                        e => e.BuildingNameTh.Replace("อาคาร", "").Trim() == objData.building && e.SubDistrict == objData.
subdistrict && e.District == objData.district && e.Province == objData.province && e.Postcode == objData.zipcode);
                    if (!IsBuildingMaster)
                    {
                        throw new APIResponseException(HttpStatusCode.Conflict, CustomErrorCode.C102, $"
ไม่มีข้อมูลBuilding ใน Master ไม่สามารถอัพเดตข้อมูลได้");

                    }
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.building)),
                        Query = objData.building
                    });
                }
```

ภาพที่ 4-25 CreateLocationMastertAsync

```
                if (!string.IsNullOrEmpty(objData.road))
                {
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.road)),
                        Query = objData.road
                    });
                }
                if (!string.IsNullOrEmpty(objData.soi))
                {
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.soi)),
                        Query = objData.soi
                    });

                }
                if (!string.IsNullOrEmpty(objData.village))
                {
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.village)),
                        Query = objData.village
                    });
                }
                if (!string.IsNullOrEmpty(objData.district))
                {
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.district)),
                        Query = objData.district
                    });
                }
                if (!string.IsNullOrEmpty(objData.subdistrict))
                {
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.subdistrict)),
                        Query = objData.subdistrict
                    });
                }
                if (!string.IsNullOrEmpty(objData.province))
                {
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.province)),
                        Query = objData.province
                    });
                }
                if (!string.IsNullOrEmpty(objData.zipcode))
                {
                    queryContainers.Add(new MatchPhraseQuery
                    {
                        Field = new Field(nameof(LocationmasterMasterModel.zipcode)),
                        Query = objData.zipcode
                    });
                }
                var boolQuery = new BoolQuery()
                {
                    Must = queryContainers
                };
                var response = await elasticUnitOfWork.CustShippingRepository.SearchAsync(boolQuery);
                if (response.Any() || response.Count() > 0)
                {
                    throw new APIResponseException(HttpStatusCode.Conflict, CustomErrorCode.C102, $"
มีอยู่แล้ว ไม่สามารถเพิ่มข้อมูลได้");
                }
                await elasticUnitOfWork.CustShippingRepository.AddAsync(objData, objData.id);

            }
            catch (Exception ex)
            {
                throw ex;
            }
        }
```

ภาพ 4-25 (ต่อ) CreateLocationMastertAsync

บทที่ 5
ผลการดำเนินโครงงาน

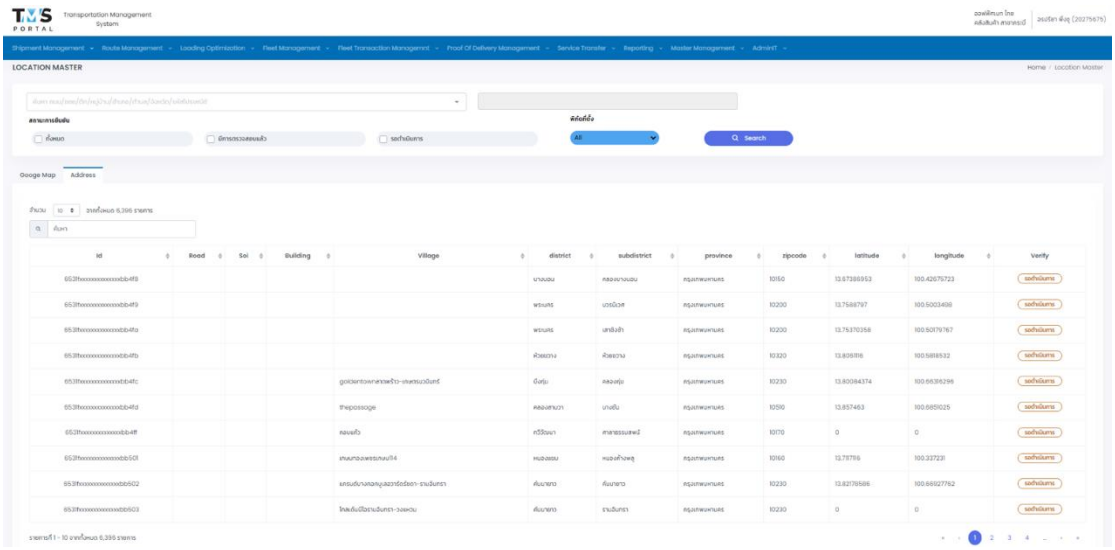
5.1 ผลการดำเนินงาน

โครงงานที่ได้จัดทำนี้เป็นการพัฒนา Web Application สำหรับการจัดการ Shipping Area Master โดยมีจุดมุ่งหมายสำหรับการพัฒนากระบวนการจัด Route plan ให้มีประสิทธิภาพ ลดความซ้ำซ้อน และมีความสะดวกรวดเร็วมากยิ่งขึ้น

5.2 การแสดงผลของ Web Application

5.2.1 หน้าหลัก

เป็นหน้าแสดงข้อมูล Shipping Area Master ดังภาพ 5-1

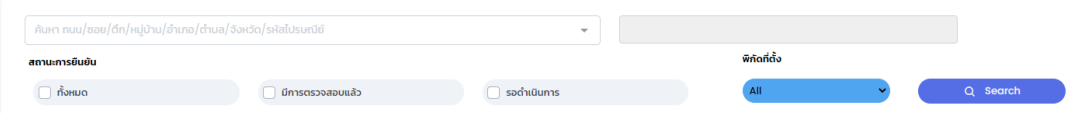


ภาพที่ 5-1 หน้าจอเว็บไซต์ Web Management

5.2.2 การคัดกรองข้อมูล Location Master

เป็นการคัดกรองข้อมูลที่จะทำการแสดงไว้ที่ การแสดงผล ที่กำหนด โดยมีข้อมูลดังนี้

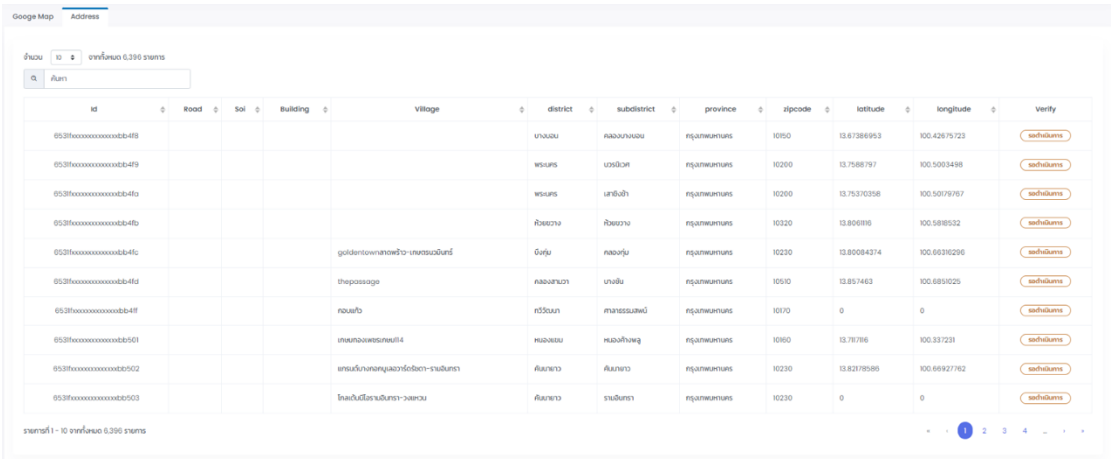
1. ค้นหาข้อมูล Shipping Area Master
2. สถานะการยืนยันการตรวจสอบ
3. พิกัดที่ตั้ง



ภาพที่ 5-2 Criteria การกรองข้อมูล

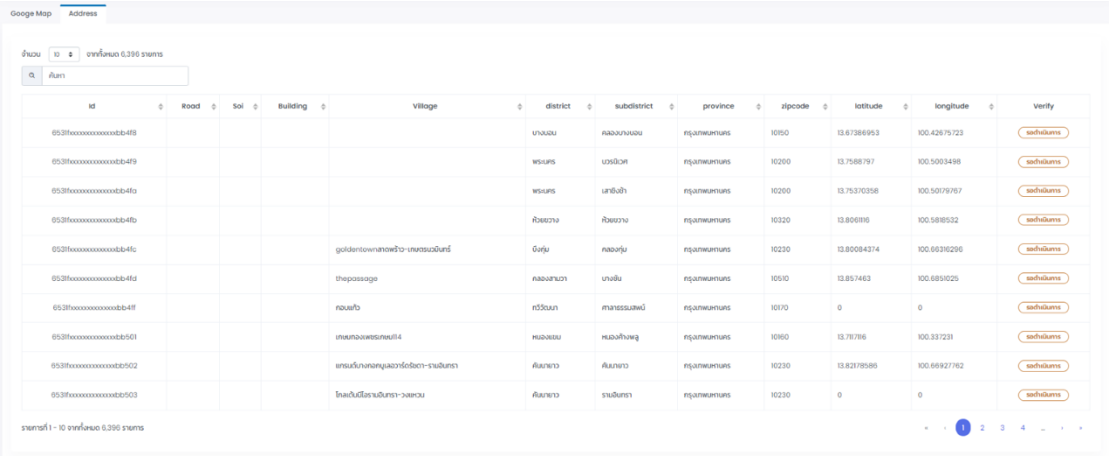
หลังจากกด Search หน้าจอแสดงผลข้อมูล Shipping Area Master สามารถแสดงผลได้ 2 รูปแบบ ดังนี้

1. Address
2. Google Map



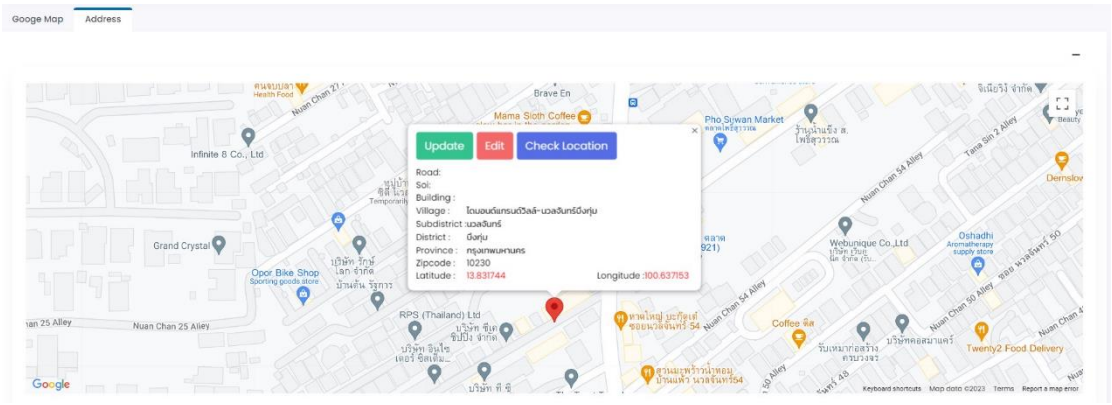
ภาพที่ 5-3 หน้าจอเลือกแสดง Display

5.2.1.1 Address



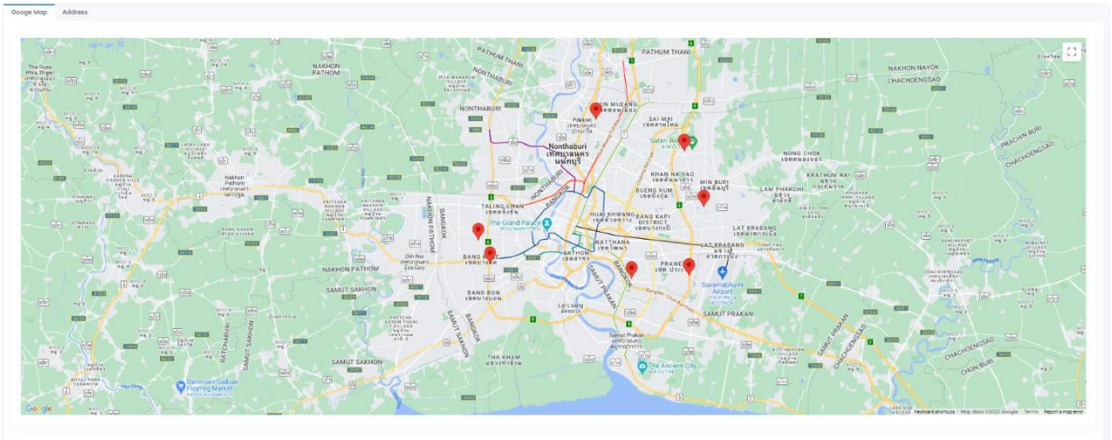
ภาพที่ 5-4 หน้าจอแสดงแบบ Address

นอกจากนี้สามารถกดดูข้อมูลสถานที่ได้ ถ้ากดที่แถวของข้อมูล จะมีแผนที่แสดงขึ้นมาให้ดูว่าข้อมูลที่เลือกอยู่ตรงไหนบนแผนที่



ภาพที่ 5-5 หน้าจอแสดงพิกัดข้อมูล

5.2.1.2 Google Map

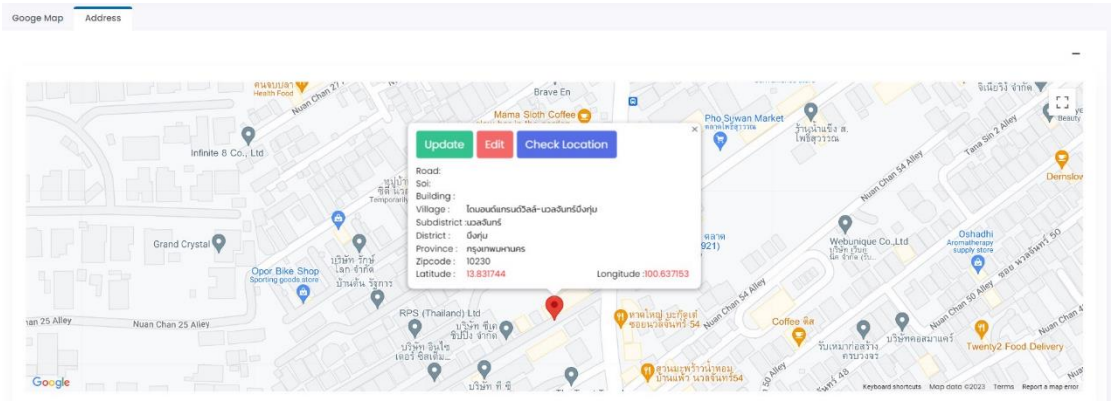


ภาพที่ 5-6 หน้าจอแสดงแบบ Google Map

5.2.3 พิกัดของข้อมูล

หลังจากกดดูข้อมูล จะแสดงพิกัดข้อมูลบนแผนที่ และจะแสดง info window ของข้อมูลนั้น โดยพิกัดข้อมูลสามารถทำงานได้ดังนี้

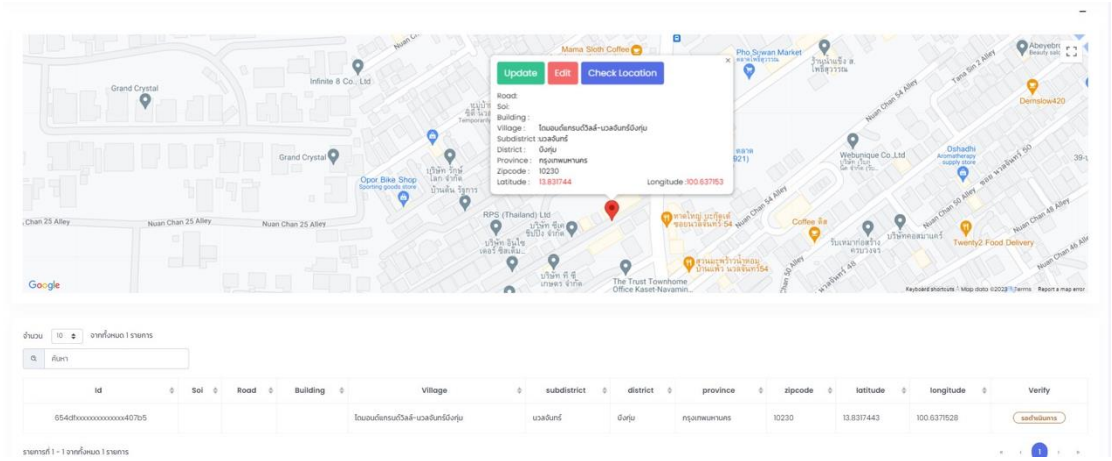
1. Update
2. Edit
3. Check Location



ภาพที่ 5-7 หน้าจอแสดงข้อมูลพิกัดบนแผนที่

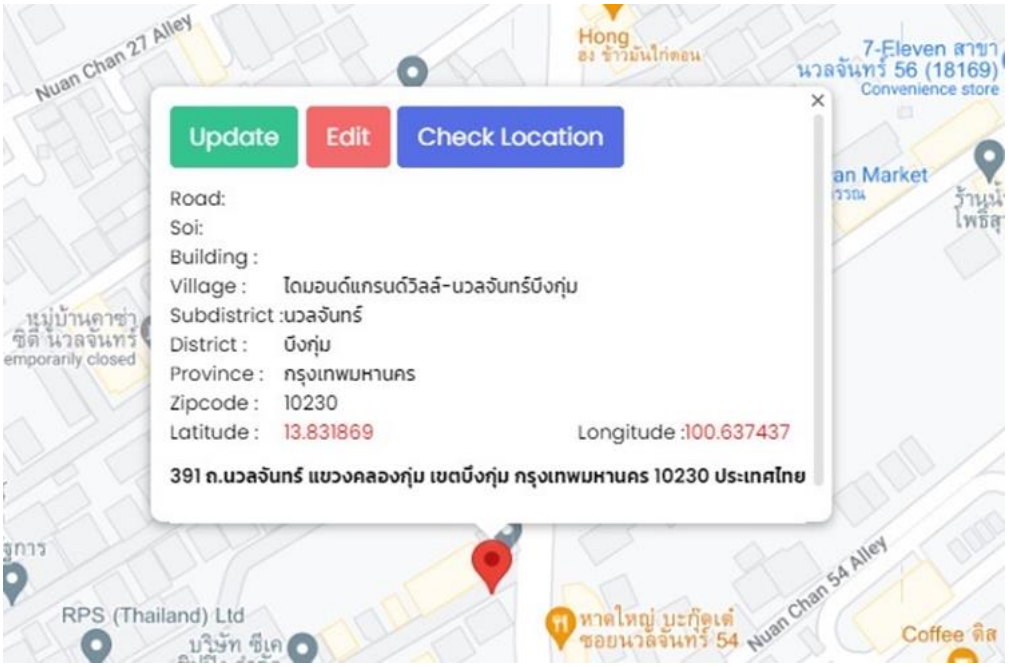
5.2.3.1 Update

ดังภาพ 5-8 คือพิกัดข้อมูลเริ่มต้น



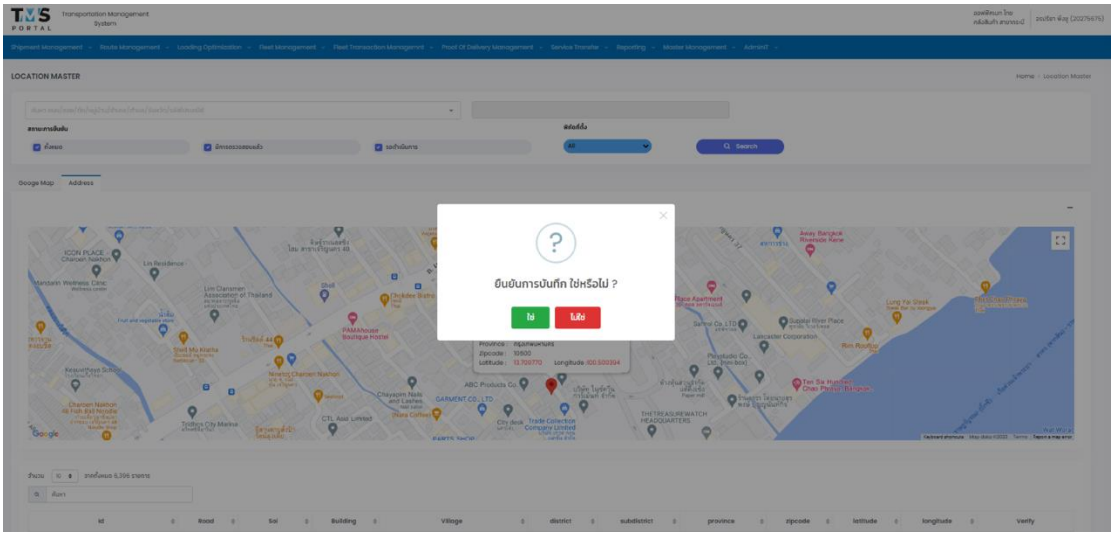
ภาพที่ 5-8 หน้าจอข้อมูลเริ่มต้น

ถ้าผู้ใช้มีการลาก Marker พิกัดนั้น จะแสดงข้อมูลที่เปลี่ยนไปใน info window



ภาพที่ 5-9 หน้าจอข้อมูลหลังมีการลาก Marker

ผู้ใช้กดปุ่ม Update จะมี Modal ขึ้นมา ดังภาพ 5-10



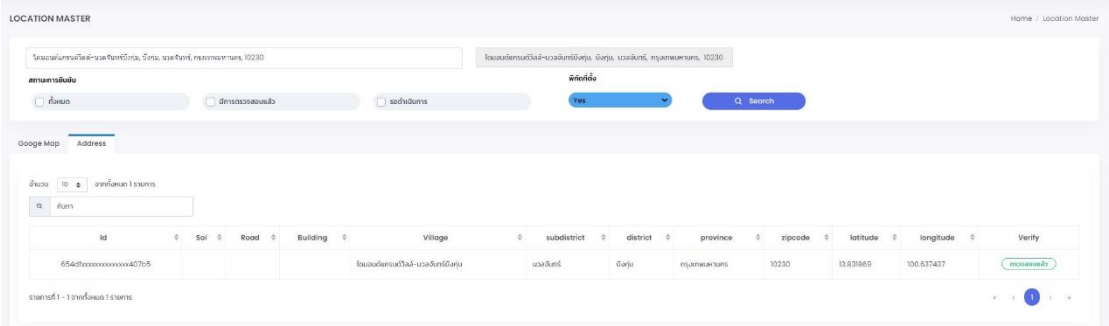
ภาพที่ 5-10 หน้าจอแสดงการรอยืนยันการ Update

แสดงผลการอัปเดตข้อมูลสำเร็จ



ภาพที่ 5-11 หน้าจอแสดงผลการอัปเดตสำเร็จ

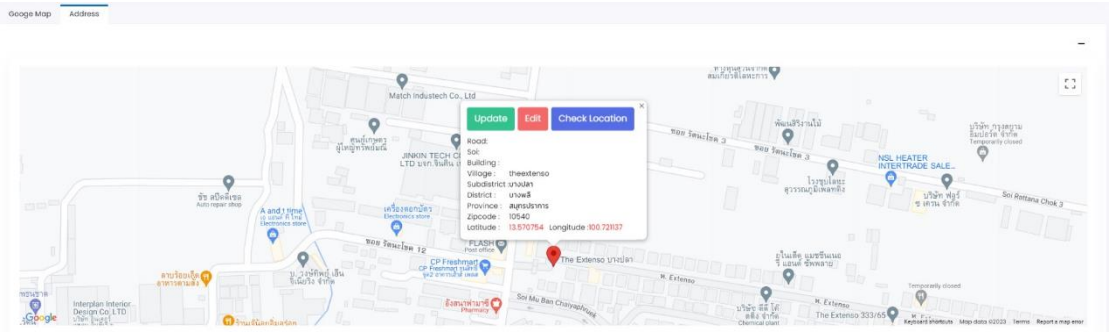
เมื่ออัปเดตข้อมูลสำเร็จ จะกลับมาแสดงตารางข้อมูล Shipping Area Master ข้อมูลที่ถูก Update จะเปลี่ยนสถานะจาก รอดำเนินการ เป็นสถานะ มีตรวจสอบแล้ว แล้ว ดังภาพ 5-12



ภาพที่ 5-12 หน้าจอแสดงการ Update

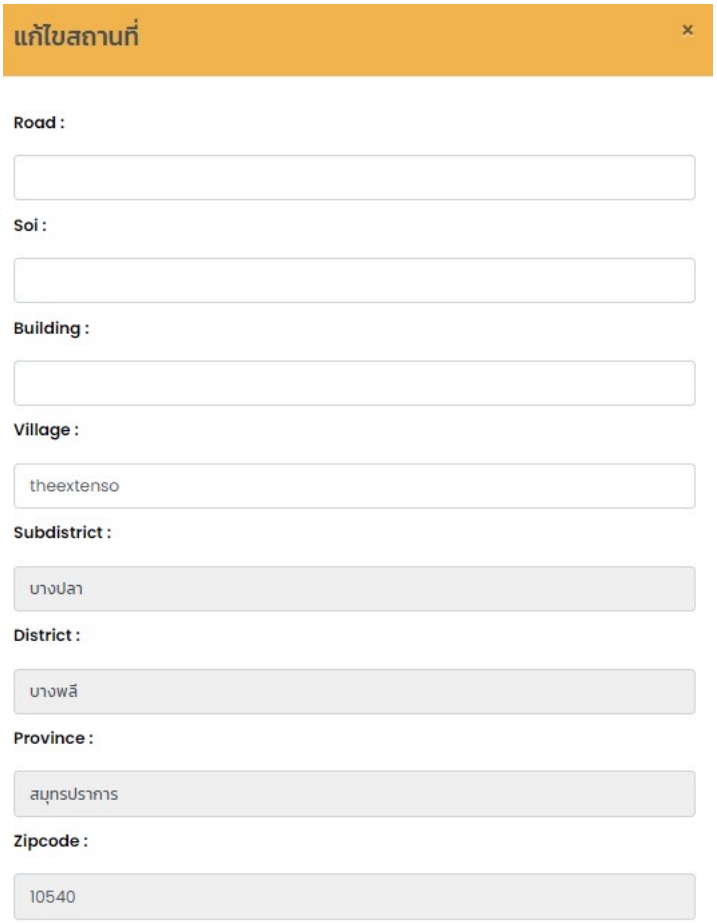
5.2.3.2 Edit

ดังภาพ 5-13 คือพิกัดข้อมูลเริ่มต้น



ภาพที่ 5-13 หน้าจอข้อมูลเริ่มต้น

เมื่อทำการกดปุ่ม Edit จะแสดง Modal ขึ้นมาดังภาพ 5-14 ให้แสดงสามารถแก้ไขข้อมูลได้



ภาพที่ 5-14 หน้าจอแสดงการแก้ไขข้อมูล

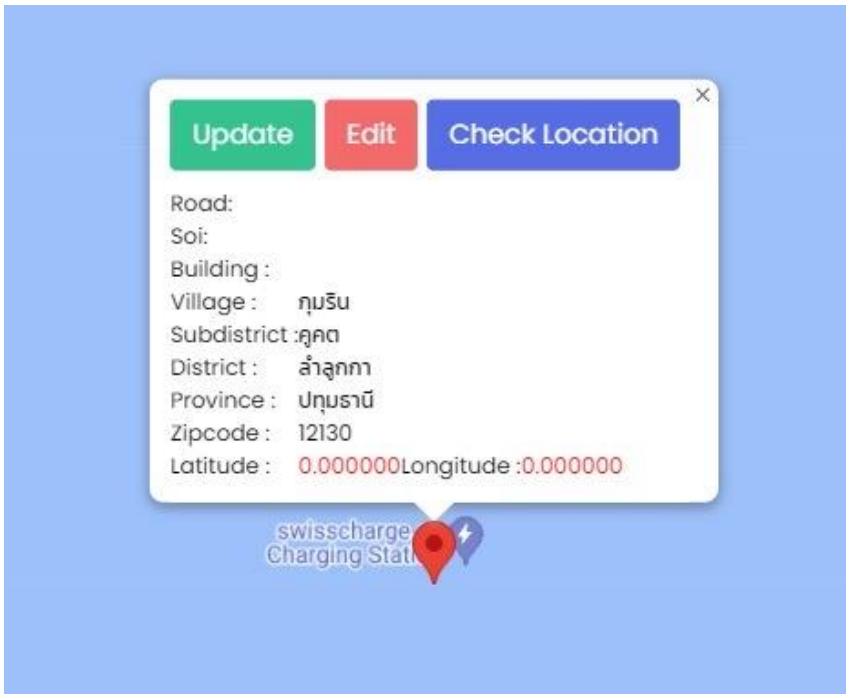
แสดงผลการแก้ไขข้อมูลสำเร็จ



ภาพที่ 5-15 หน้าจอแสดงผลการแก้ไขมูลสำเร็จ

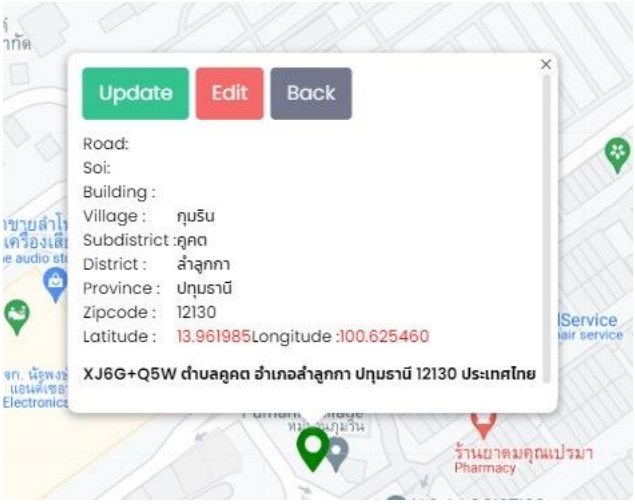
5.2.3.3 Check Location

ดังภาพ 5-16 คือพิกัดข้อมูลเริ่มต้น



ภาพที่ 5-16 หน้าจอข้อมูลเริ่มต้น

ผู้ใช้เมื่อกดปุ่ม Check Location Web จะเรียกใช้ Google Service โดยจะเอาข้อมูลที่เลือกมาไปเอาข้อมูล ละติจูด ลองจิจูด ที่ได้มาปักพิกัดใหม่บทแผนที่ และมี Address จากพิกัดนั้นมาแสดงใน info window ดังภาพ 5-17



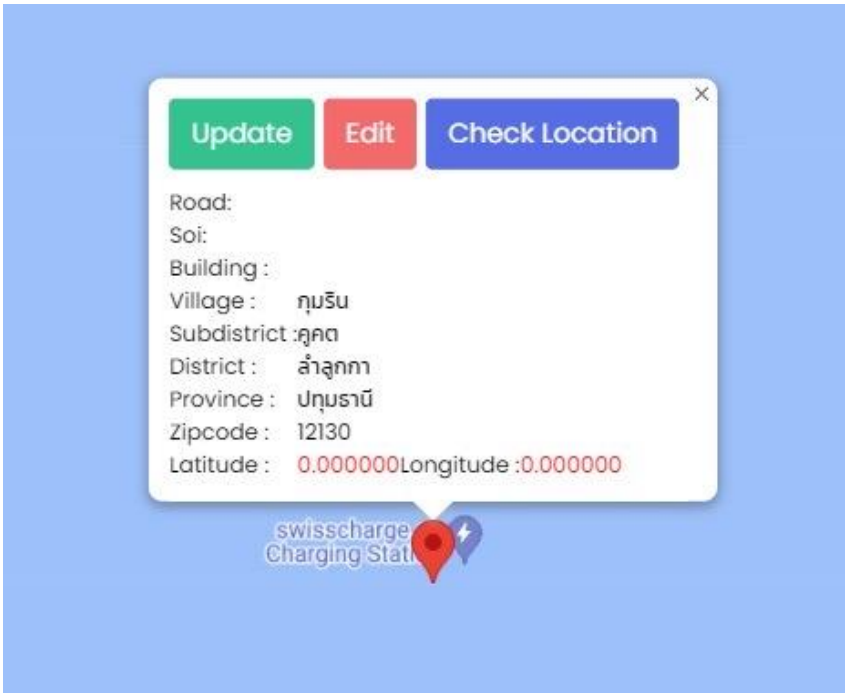
ภาพที่ 5-17 หน้าจอแสดงข้อมูล Check Location

หลังจากกดปุ่ม Check Location จะมีปุ่ม Back เพิ่มขึ้นมา ดังภาพ 5-18



ภาพที่ 5-18 หน้าจอแสดงปุ่ม Back

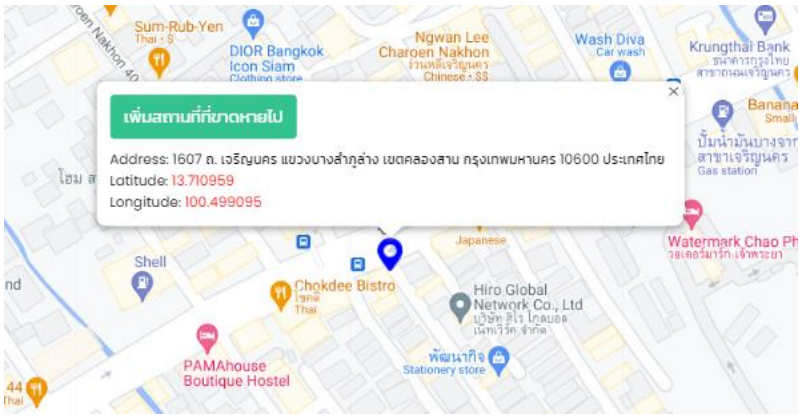
เมื่อกดปุ่ม Back จะเป็นการกลับไปจุดเริ่มต้นที่กดข้อมูลพิกัดครั้งแรก



ภาพที่ 5-19 หน้าจอแสดงข้อมูลพิกัดเริ่มต้น

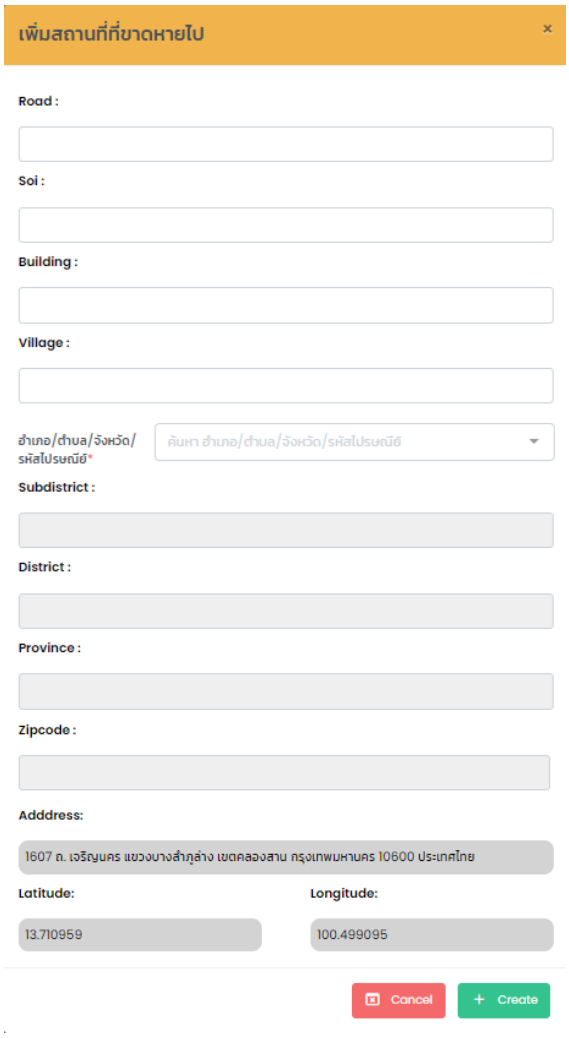
5.2.4 การเพิ่มข้อมูล

ถ้ามีการแสดงแผนที่ที่หน้า Display ไหน ถ้าผู้ใช้มีการคลิกตรงไหนก็ได้ของแผนที่ที่อยากจะเพิ่ม จะมี info window ขึ้นมา โดยรายละเอียดของพิกัดนั้นมี Address ตามพิกัดที่คลิก และมีพิกัด ละติจูด ลองจิจูด ขึ้นมาดังภาพ 5-20



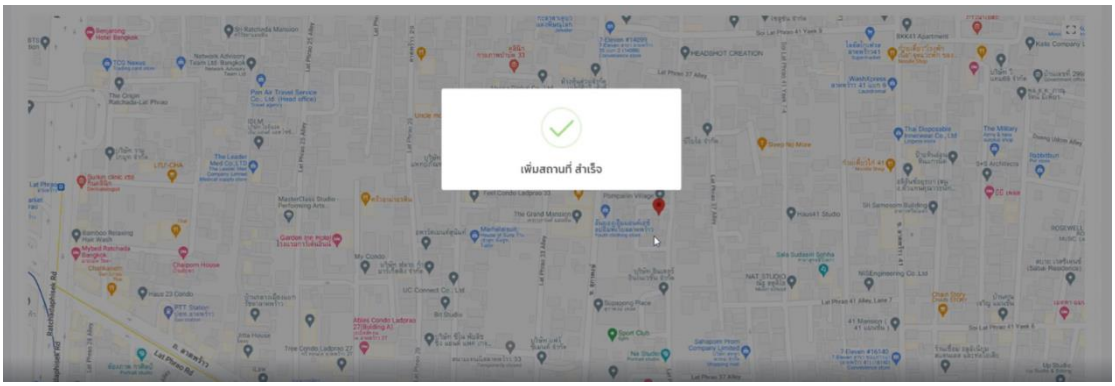
ภาพที่ 5-20 info window การกดที่แผนที่

และถ้าผู้ใช้สามารถกดปุ่ม เพิ่มสถานที่ที่ขาดหายไป จะมี Modal ขึ้นมาดังภาพ 5-21 ให้แสดงสามารถเพิ่มข้อมูลได้



ภาพที่ 5-21 หน้าจอแสดงการเพิ่มข้อมูล

แสดงผลการเพิ่มข้อมูลสำเร็จ



ภาพที่ 5-22 หน้าจอแสดงผลการเพิ่มข้อมูลสำเร็จ

บทที่ 6
บทสรุปและแนวทางในการพัฒนาต่อ

6.1 สรุปผลการดำเนินงาน

6.1.1 ได้ Web Application ที่ใช้ในการจัดการ Shipping Area Master ที่ตอบโจทย์กับทางทีม Route planning ในการจัด Route plan โดย Web Application สามารถทำงานได้ดังนี้

6.1.1.1 Web Application สามารถ เพิ่ม แก้ไข ข้อมูล Shipping Area Master ได้

6.1.1.1 Web Application สามารถเช็คข้อมูล Shipping Area Master ว่าถูกต้องได้

6.1.1.2 Web Application สามารถคัดกรองข้อมูล Shipping Area Master ได้ว่าข้อมูลไหนยังไม่ผ่านการตรวจสอบ เพื่อให้ให้ได้ข้อมูลที่มี พร้อมใช้งานตลอด

6.1.2 ได้ Service Python ที่เอาไว้ใช้งานในการจัดการข้อมูลที่อยู่ลูกค้าให้มีความละเอียดและถูกต้องมากขึ้น โดย Service สามารถทำงานได้ดังนี้

6.1.2.1 Service สามารถสกัดรายละเอียดของ Road Soi Building Village ได้

6.1.2.2 Service สามารถตัดแบ่งที่อยู่ลูกค้าได้อย่างถูกต้อง

6.1.2.3 Service สามารถตรวจสอบความถูกต้องของ Road Soi Building ได้

6.2 ปัญหาที่พบระหว่างการดำเนินงาน

6.2.1 ข้อมูล Shipping Area Master จะอยู่ที่ server ของ Elastics แล้ว server สามารถใช้งานได้บางครั้ง บางครั้งก็ไม่สามารถทำงานได้ ส่งผลให้ Web Application ไม่สามารถไปดึงข้อมูลจาก Elastics มาได้

6.2.2 การออกแบบหน้าเว็บ และความต้องการที่ได้จัดทำยังไม่ชัดเจนพอทำให้ต้องเสียเวลาในการปรับปรุงแก้ไขตลอดเวลา

6.2.3 ข้อมูลที่จะเอาไว้ตรวจสอบว่า ข้อมูลที่จะนำมาเป็น Shipping Area Master มีความถูกต้องจริงหรือไม่ มีข้อมูลไม่เพียงพอในการตรวจสอบ

6.2.4 สิทธิ์การใช้งานข้อมูลที่จะต้องใช้ในโครงงานนี้ บางครั้งไม่มีสิทธิ์ในการเข้าดู ทำให้ทำงานมีความไม่เข้าใจในบางจุด

6.3 แนวทางแก้ปัญหา

6.3.1 เปลี่ยน Server ที่จะเก็บข้อมูล Elastics เพื่อให้สามารถได้ตลอดไม่ติดขัดในการเรียกใช้

6.3.2 ติดต่อหน่วยงานที่มีข้อมูลเกี่ยวกับข้อมูลที่จะเอาไว้ตรวจสอบ และรวบรวมมาให้ได้มากสุดเพื่อให้ได้ข้อมูลที่ต้องมากขึ้น

6.3.3 มีการรายงานกับทางพี่เลี้ยงตลอดทุกสัปดาห์ เพื่อรับ Feedback มาแก้ไขให้ทัน

6.3.4 เปิดสิทธิ์ให้เข้าถึงข้อมูล เพื่อทำให้เกิดความเข้าใจที่ตรงกัน

6.4 แนวทางในการพัฒนาต่อ

6.4.1 พัฒนาให้ Web Application มีความเสถียรในการใช้งานให้มากกว่าเดิม

6.4.2 สามารถปรับแต่งหน้าเว็บให้ดูใช้งานได้ง่ายมากกว่านี้ เนื่องจากปัจจุบันยังมีบางฟังก์ชันที่ดูใช้งานยาก

6.4.3 Service Python ที่ทำขึ้นมาสามารถนำไปใช้ระบบอื่นที่ไม่ใช่ ระบบ Transportation Management System (TMS) ได้

6.4.4 ข้อมูล Shipping Area Master ที่ได้มาสามารถนำไปใช้จัดการกับ Feature อื่นที่ไม่ใช่ Feature การจัด Route plan อย่างเดียว

บรรณานุกรม

[1] Vue [สืบค้นวันที่ 9 พฤษภาคม 2566][ออนไลน์]
ที่มา https://appmaster.io/th/blog/vuejs-khuue-aair

[2] NodeJS ทำงานอย่างไร ? [สืบค้นวันที่ 9 พฤษภาคม 2566][ออนไลน์]
ที่มา https://blog.openlandscape.cloud/nodejs

[3] C# คืออะไร [สืบค้นวันที่ 9 พฤษภาคม 2566][ออนไลน์]
ที่มา https://www.mindphp.com

[4] SQL คืออะไร เหตุใดจึง [สืบค้นวันที่ 13 มิถุนายน 2566][ออนไลน์]
ที่มา https://appmaster.io/th/blog/sql-khuue-aair

[5] Maps JavaScript API [สืบค้นวันที่ 5 กรกฎาคม 2566][ออนไลน์]
ที่มา https://developers.google.com/maps/documentation/javascript/overview

[6] คุณสมบัติของภาษา [สืบค้นวันที่ 5 กรกฎาคม 2566][ออนไลน์]
ที่มา http://marcuscode.com/lang/javascript/introducing-to-javascript

[7] PyThaiNLP Library NLP ภาษาไทย [สืบค้นวันที่ 28 กรกฎาคม 2566][ออนไลน์]
ที่มา https://www.bualabs.com/archives/3234/what-is-pythainlp-tutorial-teach-basic-how-to-use-pythainlp-library-nlp-in-python-pythainlp-ep-1/

[8] PostgREST Web Server & RESTful API[สืบค้นวันที่ 28 กรกฎาคม 2566][ออนไลน์]
ที่มา https://www.linuxadictos.com/th/postgrest-un-webserver-y-api-restful-para-cualquier-base-de-datos-postgresql.html

[9] Place API [สืบค้นวันที่ 3 สิงหาคม 2566][ออนไลน์]
ที่มา https://developers.google.com/maps/documentation/places/web-service

[10] Geocoding API [สืบค้นวันที่ 3 สิงหาคม 2566][ออนไลน์]
ที่มา https://developers.google.com/maps/documentation/geocoding

[11] Maps JavaScript API [สืบค้นวันที่ 3 สิงหาคม 2566][ออนไลน์]
ที่มา https://developers.google.com/maps/documentation/javascript

[12] Info Windows [สืบค้นวันที่ 1 กันยายน 2566][ออนไลน์]
ที่มา https://developers.google.com/maps/documentation/javascript/infowindows

[13] Markers [สืบค้นวันที่ 21 กันยายน 2566][ออนไลน์]
ที่มา https://developers.google.com/maps/documentation/javascript/markers